

LAPORAN TUGAS PRAKTIKUM SISTEM OPERASI



**Dosen Pengampu Matakuliah :
Triawan Adi Cahyanto, M.K**

**Disusun Oleh :
Muhammad Gufron (2410651053)**

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
TAHUN AKADEMIK 2024/2025**

KATA PENGANTAR

Puji syukur kehadirat Allah SWT yang telah memberikan rahmat dan hidayah-Nya sehingga kami dapat menyelesaikan tugas makalah yang berjudul “memory” ini tepat waktu.

Adapun tujuan dari penulisan makalah ini adalah untuk memenuhi tugas dosen pada mata kuliah Sistem Operasi. Selain itu, makalah ini juga bertujuan untuk menambah wawasan bagi para pembaca dan juga bagi penulis.

Kami mengucapkan terima kasih kepada Triawan Adi Cahyanto, M.K

selaku dosen mata kuliah Pengembangan Diri yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan bidang studi yang kami tekuni.

Kami juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga kami dapat menyelesaikan makalah ini. Kami menyadari, makalah yang kami tulis ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun akan kami nantikan demi kesempurnaan makalah ini.

DAFTAR ISI

BAB 1	4
PENDAHULUAN	4
1.1 LATAR BELAKANG	4
1.2 TUJUAN PRAKTIKUM	4
1.3 SPESIFIKASI SISTEM.....	4
BAB II	6
DASAR TEORI.....	6
2.1 MEMORY MANAGEMENT.....	6
2.2 CONTIGUOUS ALLOCATION.....	6
2.3 PAGING.....	6
2.4 PAGE TABLE STRUCTURES.....	6
2.5 SWAPPING	7
2.6 ARCHITECTURE	7
BAB III.....	8
JAWABAN TUGAS PRAKTIKUM	8
3.1 TUGAS 1.....	8
3.2 TUGAS 2.....	11
3.3 TUGAS 3.....	20
3.4 TUGAS 4.....	42
3.5 TUGAS 5.....	48
3.6 TUGAS 6.....	55
3.7 TUGAS 7.....	Error! Bookmark not defined.
BAB IV	60
ANALISIS DAN PEMBAHASAN.....	60
4.1 ANALISIS TUGAS 1	60
4.2 ANALISIS TUGAS 2	63
4.3 ANALISIS TUGAS 3	63
4.4 ANALISIS TUGAS 4	66
4.5 ANALISIS TUGAS 5	68
4.6 ANALISIS TUGAS 6	69
BAB V	72
KESIMPULAN DAN SARAN	72
DAFTAR PUSTAKA	Error! Bookmark not defined.

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Manajemen memori merupakan komponen kritis dalam sistem operasi yang menentukan kinerja dan stabilitas sistem. Praktikum ini dirancang untuk menganalisis dan mengimplementasikan berbagai teknik pengelolaan memori pada lingkungan Linux. Tujuannya meliputi pemahaman layout memori, simulasi algoritma alokasi kontigu dan paging, serta evaluasi mekanisme swapping dan struktur tabel halaman bertingkat. Dengan demikian, diharapkan mahasiswa dapat menginternalisasi konsep-konsep fundamental sekaligus menerapkannya dalam skenario nyata untuk optimasi sistem.

1.2 TUJUAN PRAKTIKUM

ujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Menganalisis tata letak memori (memory layout) pada proses di sistem Linux.
2. Mengimplementasikan dan membandingkan algoritma alokasi memori kontigu (First Fit, Best Fit, Worst Fit, Next Fit).
3. Mensimulasikan sistem paging dengan algoritma penggantian halaman FIFO dan LRU.
4. Memahami struktur multi-level page table dan inverted page table serta overhead yang ditimbulkannya.
5. Mengamati mekanisme swapping dan menganalisis pengaruh strategi pemilihan proses terhadap performa sistem.

1.3 SPESIFIKASI SISTEM

Praktikum ini dilakukan pada lingkungan sistem operasi Linux. Adapun spesifikasi perangkat dan lingkungan yang digunakan adalah sebagai berikut:

- Sistem Operasi: Linux (distribusi dapat disesuaikan, misalnya Ubuntu)
- Arsitektur Prosesor: x86_64 (64-bit)
- Compiler: GCC (GNU Compiler Collection)

- Memory Fisik: ≥ 4 GB
- Swap Space: ≥ 2 GB
- **Tools Pendukung:**
- Gcc untuk kompilasi program C
- `cat /proc/[pid]/maps` untuk melihat peta memori proses
- `lscpu` `free`, `vmstat`, `swapon` untuk analisis system
- Editor teks (seperti VS Code, Nano, atau Vim) untuk penulisan kode

BAB II

DASAR TEORI

2.1 MEMORY MANAGEMENT

Manajemen memori adalah fungsi vital dalam sistem operasi yang bertugas mengelola memori utama (RAM) agar penggunaan ruang penyimpanan oleh berbagai proses dapat berjalan secara optimal. Tugas utamanya meliputi pelacakan bagian memori yang sedang digunakan dan yang bebas, serta mengalokasikan memori ke proses yang membutuhkannya dan mendelokasikannya setelah proses selesai. Tanpa manajemen yang efisien, sistem akan mengalami kesulitan dalam menjalankan banyak aplikasi secara bersamaan (multiprogramming).

2.2 CONTIGUOUS ALLOCATION

Contiguous allocation adalah metode pengalokasian memori di mana suatu proses ditempatkan pada satu blok memori yang berurutan (kontigu). Metode ini sederhana untuk diimplementasikan karena alamat memori proses dapat dihitung dengan mudah menggunakan base dan limit register. Namun, contiguous allocation memiliki beberapa kelemahan, terutama terjadinya fragmentasi eksternal, yaitu kondisi di mana terdapat ruang kosong yang tersebar tetapi tidak cukup besar untuk dialokasikan ke proses baru.

2.3 PAGING

Paging adalah teknik manajemen memori yang membagi memori fisik menjadi blok-blok berukuran tetap yang disebut frame, serta memori logis menjadi page dengan ukuran yang sama. Dengan paging, proses tidak perlu ditempatkan secara kontigu di memori fisik, sehingga masalah fragmentasi eksternal dapat dihilangkan. Sistem operasi menggunakan page table untuk memetakan page ke frame yang sesuai. Meskipun paging dapat meningkatkan efisiensi penggunaan memori, teknik ini menambah overhead karena diperlukan mekanisme translasi alamat dari virtual ke fisik.

2.4 PAGE TABLE STRUCTURES

Page table structures digunakan untuk menyimpan informasi pemetaan antara page virtual dan frame fisik. Pada sistem dengan ruang alamat besar, penggunaan single-level page table menjadi tidak efisien karena membutuhkan memori yang sangat besar. Oleh karena itu, dikembangkan struktur page table bertingkat seperti two-level dan three-level page table. Selain itu, inverted page table juga digunakan untuk mengurangi kebutuhan memori dengan membuat satu tabel global berdasarkan frame fisik.

2.5 SWAPPING

Swapping adalah teknik di mana sebuah proses dapat dikeluarkan sementara dari memori utama ke penyimpanan sekunder (seperti disk) dan kemudian dibawa kembali ke memori untuk melanjutkan eksekusi. Prosedur ini sangat berguna ketika total memori yang dibutuhkan oleh semua proses aktif melebihi kapasitas fisik RAM yang tersedia. Dengan melakukan swap out pada proses yang sedang menganggur dan swap in pada proses yang siap dieksekusi, sistem dapat menjalankan lebih banyak aplikasi daripada yang diizinkan oleh batas fisik memori.

2.6 ARCHITECTURE

Swapping adalah mekanisme manajemen memori di mana proses atau bagian dari proses dipindahkan sementara dari memori utama ke media penyimpanan sekunder, seperti hard disk, untuk memberi ruang bagi proses lain. Teknik ini memungkinkan sistem menjalankan lebih banyak proses dibandingkan kapasitas memori fisik yang tersedia. Namun, swapping dapat menurunkan kinerja sistem karena waktu akses ke media penyimpanan sekunder jauh lebih lambat dibandingkan memori utama. Oleh karena itu, sistem operasi harus mengatur swapping secara optimal agar tidak menyebabkan overhead yang berlebihan.

BAB III

JAWABAN TUGAS PRAKTIKUM

1.1 TUGAS 1 Analisis Memory Layout

1.1.1 Deskripsi Tugas

Program ini dibuat untuk mengamati dan menganalisis tata letak memori proses (memory layout) pada sistem operasi Linux menggunakan bahasa C. Melalui program ini, ditunjukkan secara langsung alamat memori dari berbagai segmen utama, yaitu text segment (fungsi main dan fungsi lainnya), data segment (variabel global, array statis, dan string literal), heap (alokasi dinamis menggunakan malloc), serta stack (variabel lokal dan pemanggilan rekursif). Selain itu, program menampilkan perubahan alamat stack pada setiap level rekursi untuk menunjukkan arah pertumbuhan stack. Hasil alamat tersebut kemudian dibandingkan dengan memory map proses yang diperoleh dari file `/proc/[pid]/maps` guna memverifikasi kesesuaian segmen dan permission memori.

1.1.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
/* prototype */
int main();

/* variabel global */
int global_var = 100;
int static_array[3] = {10, 20, 30};
const char *text = "Main Memory Praktikum";
/* fungsi biasa */
void func1() {}
void func2() {}
void func3() {}
/* fungsi rekursif */
void recursive(int level) {
    int x = level;
```



```

printf("  Level %d local_var address: %p value: %d\n",
       level, (void*)&x, x);
if (level < 2)
    recursive(level + 1);
}

void print_memory() {
    int local = 5;
    int *heap1 = malloc(sizeof(int));
    int *heap2 = malloc(sizeof(int));
    int *dyn_array = malloc(3 * sizeof(int));
    /* isi nilai */
    *heap1 = 50;
    *heap2 = 75;
    dyn_array[0] = 1;
    printf("\n=== ENHANCED MEMORY LAYOUT ===\n\n");
    printf("1. Text Segment:\n");
    printf("  main() : %p\n", (void*)main);
    printf("  func1() : %p\n", (void*)func1);
    printf("  func2() : %p\n", (void*)func2);
    printf("  func3() : %p\n\n", (void*)func3);
    printf("2. Data Segment:\n");
    printf("  global_var address: %p value: %d\n",
           (void*)&global_var, global_var);
    printf("  static_array[0] address: %p value: %d\n",
           (void*)&static_array[0], static_array[0]);
    printf("  string literal address: %p value: %s\n\n",
           (void*)text, text);
    printf("3. Heap:\n");
    printf("  heap1 address: %p value: %d\n",

```

```

        (void*)heap1, *heap1);
printf(" heap2 address: %p value: %d\n",
        (void*)heap2, *heap2);
printf(" dyn_array[0] address: %p value: %d\n\n",
        (void*)&dyn_array[0], dyn_array[0]);
printf("4. Stack:\n");
printf(" local address: %p value: %d\n\n",
        (void*)&local, local);
printf("4. Stack (Recursive Calls):\n");
recursive(0);
free(heap1);
free(heap2);
free(dyn_array);
}
int main() {
    printf("PID: %d\n", getpid());
    print_memory();
    printf("\nTekan Enter untuk melihat memory map...");
    getchar();
    char cmd[100];
    sprintf(cmd, "cat /proc/%d/maps", getpid());
    system(cmd);
    return 0;
}

```

1.1.3 Screenshots Output

```

gufro@MSI:~/praktikum-memory/architecture$ nano address_demo.c
gufro@MSI:~/praktikum-memory/architecture$ gcc -o address_demo address_demo.c
gufro@MSI:~/praktikum-memory/architecture$ ./address_demo
PID: 2579

=== ENHANCED MEMORY LAYOUT ===

1. Text Segment:
   main() : 0x58e161dab54b
   func1() : 0x58e161dab249
   func2() : 0x58e161dab254
   func3() : 0x58e161dab25f

2. Data Segment:
   global_var address: 0x58e161dae010 value: 100
   static_array[0] address: 0x58e161dae018 value: 10
   string literal address: 0x58e161dac008 value: Main Memory Praktikum

3. Heap:
   heap1 address: 0x58e19bfc56b0 value: 50
   heap2 address: 0x58e19bfc56d0 value: 75
   dyn_array[0] address: 0x58e19bfc56f0 value: 1

4. Stack:
   local address: 0x7ffc325e87fc value: 5

4. Stack (Recursive Calls):
   Level 0 local_var address: 0x7ffc325e87d4 value: 0
   Level 1 local_var address: 0x7ffc325e87a4 value: 1
   Level 2 local_var address: 0x7ffc325e8774 value: 2

Tekan Enter untuk melihat memory map...
58e161daa000-58e161dab000 r--p 00000000 08:30 21529 /home/gufro/praktikum-memory/architecture/address_demo
58e161dab000-58e161dac000 r-xp 00001000 08:30 21529 /home/gufro/praktikum-memory/architecture/address_demo
58e161dac000-58e161dad000 r--p 00002000 08:30 21529 /home/gufro/praktikum-memory/architecture/address_demo
58e161dad000-58e161dae000 r--p 00002000 08:30 21529 /home/gufro/praktikum-memory/architecture/address_demo
58e161dae000-58e161daf000 r-wp 00003000 08:30 21529 /home/gufro/praktikum-memory/architecture/address_demo
58e19bfc5000-58e19bfc6000 r-wp 00000000 00:00 0 [heap]
7db30a600000-7db30a620000 r--p 00000000 08:30 46696 /usr/lib/x86_64-linux-gnu/libc.so.6
7db30a620000-7db30a700000 r-xp 00028000 08:30 46696 /usr/lib/x86_64-linux-gnu/libc.so.6
7db30a700000-7db30a7ff000 r--p 001b0000 08:30 46696 /usr/lib/x86_64-linux-gnu/libc.so.6
7db30a7ff000-7db30a803000 r--p 001fc000 08:30 46696 /usr/lib/x86_64-linux-gnu/libc.so.6
7db30a803000-7db30a805000 r-wp 00222000 08:30 46696 /usr/lib/x86_64-linux-gnu/libc.so.6
7db30a805000-7db30a812000 r-wp 00000000 00:00 0
7db30a97d000-7db30a980000 r-wp 00000000 00:00 0
7db30a980000-7db30a989000 r-wp 00000000 00:00 0
7db30a989000-7db30a9ba000 r--p 00000000 08:30 46693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7db30a9ba000-7db30a9b5000 r-xp 00001000 08:30 46693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7db30a9b5000-7db30a9b9000 r--p 0002c000 08:30 46693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7db30a9b9000-7db30a9c1000 r--p 00036000 08:30 46693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7db30a9c1000-7db30a9c3000 r-wp 00038000 08:30 46693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffc325c9000-7ffc325eb000 r-wp 00000000 00:00 0 [stack]
7ffc325f9000-7ffc325fd000 r--p 00000000 00:00 0 [vvar]
7ffc325fd000-7ffc325ff000 r-xp 00000000 00:00 0 [vdso]
gufro@MSI:~/praktikum-memory/architecture$

```

1.1.4 Penjelasan Implementasi

Program ini mengimplementasikan eksplorasi layout memori proses pada sistem Linux dengan menampilkan alamat dari setiap segmen memori utama. Segmen text ditunjukkan melalui alamat fungsi, segmen data melalui variabel global, array statis, dan string literal, sedangkan segmen heap diperlihatkan menggunakan alokasi dinamis malloc. Segmen stack diamati melalui variabel lokal dan pemanggilan fungsi rekursif untuk menunjukkan perubahan alamat pada setiap level. Program juga menampilkan PID proses dan membaca file /proc/[pid]/maps untuk membandingkan alamat yang dicetak dengan peta memori aktual, sehingga membantu memahami pemisahan dan karakteristik setiap segmen memori.

1.2 TUGAS 2 Implementasi Memory Allocator

1.2.1 Deskripsi Tugas

1.2.2 Source Code

```

#include <stdio.h>

#include <time.h>

#define MAX_BLOCKS 100

typedef struct {
    int size;

```

```

    int free;    // 1 = free, 0 = allocated
    int process_id;
} Block;

Block memory[MAX_BLOCKS];
int block_count = 1;
int last_alloc_index = 0;

/* ===== INIT MEMORY ===== */
void init_memory(int total_size) {
    memory[0].size = total_size;
    memory[0].free = 1;
    memory[0].process_id = -1;
    block_count = 1;
    last_alloc_index = 0;
}

/* ===== DISPLAY ===== */
void display_memory() {
    printf("\nMemory State:\n");
    for (int i = 0; i < block_count; i++) {
        if (memory[i].free)
            printf("Block %d: FREE | Size = %d\n", i, memory[i].size);
        else
            printf("Block %d: P%d | Size = %d\n", i, memory[i].process_id, memory[i].size);
    }
}

/* ===== DEALLOCATE ===== */
void deallocate(int pid) {

```

```

for (int i = 0; i < block_count; i++) {
    if (!memory[i].free && memory[i].process_id == pid) {
        memory[i].free = 1;
        memory[i].process_id = -1;
        return;
    }
}
}

/* ===== FIRST FIT ===== */
int first_fit(int pid, int size) {
    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {

            if (memory[i].size > size) {
                for (int j = block_count; j > i + 1; j--)
                    memory[j] = memory[j - 1];

                memory[i + 1].size = memory[i].size - size;
                memory[i + 1].free = 1;
                memory[i + 1].process_id = -1;
                block_count++;
            }

            memory[i].size = size;
            memory[i].free = 0;
            memory[i].process_id = pid;
            return 1;
        }
    }
}

```

```

    return 0;
}

/* ===== BEST FIT ===== */
int best_fit(int pid, int size) {
    int best = -1;

    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {
            if (best == -1 || memory[i].size < memory[best].size)
                best = i;
        }
    }

    if (best == -1) return 0;

    if (memory[best].size > size) {
        for (int j = block_count; j > best + 1; j--)
            memory[j] = memory[j - 1];

        memory[best + 1].size = memory[best].size - size;
        memory[best + 1].free = 1;
        memory[best + 1].process_id = -1;
        block_count++;
    }

    memory[best].size = size;
    memory[best].free = 0;
    memory[best].process_id = pid;
    return 1;
}

```

```

}

/* ===== WORST FIT ===== */
int worst_fit(int pid, int size) {
    int worst = -1;

    for (int i = 0; i < block_count; i++) {
        if (memory[i].free && memory[i].size >= size) {
            if (worst == -1 || memory[i].size > memory[worst].size)
                worst = i;
        }
    }

    if (worst == -1) return 0;

    if (memory[worst].size > size) {
        for (int j = block_count; j > worst + 1; j--)
            memory[j] = memory[j - 1];

        memory[worst + 1].size = memory[worst].size - size;
        memory[worst + 1].free = 1;
        memory[worst + 1].process_id = -1;
        block_count++;
    }

    memory[worst].size = size;
    memory[worst].free = 0;
    memory[worst].process_id = pid;
    return 1;
}

```

```

/* ===== NEXT FIT ===== */
int next_fit(int pid, int size) {
    int i = last_alloc_index;
    int scanned = 0;

    while (scanned < block_count) {
        if (memory[i].free && memory[i].size >= size) {

            if (memory[i].size > size) {
                for (int j = block_count; j > i + 1; j--)
                    memory[j] = memory[j - 1];

                memory[i + 1].size = memory[i].size - size;
                memory[i + 1].free = 1;
                memory[i + 1].process_id = -1;
                block_count++;
            }

            memory[i].size = size;
            memory[i].free = 0;
            memory[i].process_id = pid;

            last_alloc_index = i;
            return 1;
        }

        i = (i + 1) % block_count;
        scanned++;
    }
}

```



```

    return 0;
}

/* ===== FRAGMENTATION ===== */
void calculate_fragmentation() {
    int total_free = 0, largest = 0, holes = 0;

    for (int i = 0; i < block_count; i++) {
        if (memory[i].free) {
            total_free += memory[i].size;
            holes++;
            if (memory[i].size > largest)
                largest = memory[i].size;
        }
    }

    float ext = (total_free > 0) ?
        (1.0 - (float)largest / total_free) * 100 : 0;

    printf("Total Free Space    : %d\n", total_free);
    printf("Largest Free Block   : %d\n", largest);
    printf("Free Holes           : %d\n", holes);
    printf("External Fragmentation: %.2f%%\n", ext);
}

/* ===== BENCHMARK ===== */
void benchmark(const char *name, int (*alloc)(int,int)) {
    init_memory(1000);
    clock_t start = clock();

```

```

    alloc(1,100);
    alloc(2,200);
    alloc(3,300);
    alloc(4,250);
    alloc(5,150);

    deallocate(2);
    deallocate(4);

    alloc(6,180);
    alloc(7,220);
    alloc(8,100);

    clock_t end = clock();

    printf("\n=== %s ===\n", name);
    printf("Execution Time: %.2f us\n",
        (double)(end-start)*1e6/CLOCKS_PER_SEC);

    calculate_fragmentation();
    display_memory();
}

/* ===== MAIN ===== */
int main() {
    benchmark("First Fit", first_fit);
    benchmark("Best Fit", best_fit);
    benchmark("Worst Fit", worst_fit);
    benchmark("Next Fit", next_fit);
    return 0;
}

```

}

1.2.3 Screenshots Output

```
gufro@MSI:~/praktikum-memory/architecture$ nano memory_allocator.c
gufro@MSI:~/praktikum-memory/architecture$ gcc memory_allocator.c -o memory_allocator
gufro@MSI:~/praktikum-memory/architecture$ ./memory_allocator
```

```
=== First Fit ===
Execution Time: 2.00 us
Total Free Space      : 50
Largest Free Block    : 30
Free Holes            : 2
External Fragmentation: 40.00%
```

```
Memory State:
Block 0: P1 | Size = 100
Block 1: P6 | Size = 180
Block 2: FREE | Size = 20
Block 3: P3 | Size = 300
Block 4: P7 | Size = 220
Block 5: FREE | Size = 30
Block 6: P5 | Size = 150
```

```
=== Best Fit ===
Execution Time: 2.00 us
Total Free Space      : 50
Largest Free Block    : 30
Free Holes            : 2
External Fragmentation: 40.00%
```

```
Memory State:
Block 0: P1 | Size = 100
Block 1: P6 | Size = 180
Block 2: FREE | Size = 20
Block 3: P3 | Size = 300
Block 4: P7 | Size = 220
Block 5: FREE | Size = 30
Block 6: P5 | Size = 150
```

```
=== Worst Fit ===
Execution Time: 1.00 us
Total Free Space      : 170
Largest Free Block    : 100
Free Holes            : 2
External Fragmentation: 41.18%
```

```
Memory State:
Block 0: P1 | Size = 100
Block 1: P8 | Size = 100
Block 2: FREE | Size = 100
Block 3: P3 | Size = 300
Block 4: P6 | Size = 180
```

```
Block 3: P3 | Size = 300
Block 4: P6 | Size = 180
Block 5: FREE | Size = 70
Block 6: P5 | Size = 150
```

```
=== Next Fit ===
Execution Time: 1.00 us
Total Free Space      : 50
Largest Free Block    : 30
Free Holes            : 2
External Fragmentation: 40.00%
```

```
Memory State:
Block 0: P1 | Size = 100
Block 1: P6 | Size = 180
Block 2: FREE | Size = 20
Block 3: P3 | Size = 300
Block 4: P7 | Size = 220
Block 5: FREE | Size = 30
Block 6: P5 | Size = 150
```

1.2.4 Penjelasan Emplementasi

Program ini mengimplementasikan simulasi contiguous memory allocation untuk membandingkan algoritma First Fit, Best Fit, Worst Fit, dan Next Fit. Memori direpresentasikan sebagai kumpulan blok yang memiliki ukuran, status bebas/terpakai, dan ID proses. Saat alokasi, blok bebas dapat dipecah jika ukurannya lebih besar dari kebutuhan proses. Proses deallocasi mengosongkan kembali blok tertentu. Program menjalankan skenario alokasi dan dealokasi yang sama untuk setiap algoritma, kemudian mengukur waktu

eksekusi serta menghitung external fragmentation berdasarkan total ruang bebas, blok terbesar, dan jumlah hole. Hasil simulasi menampilkan kondisi akhir memori untuk analisis efisiensi tiap algoritma.

1.3 TUGAS 3 Advanced Paging System

1.3.1 Deskripsi Tugas

Tugas ini bertujuan untuk mensimulasikan sistem paging pada manajemen memori dengan menerapkan algoritma page replacement FIFO (First In First Out) dan LRU (Least Recently Used). Program dirancang untuk menampilkan proses alokasi halaman ke frame fisik, mendeteksi page fault dan page hit, serta mencatat statistik kinerja seperti hit rate, fault rate, dan jumlah penulisan ke disk. Selain itu, program menyediakan fitur visualisasi page table, physical memory, skenario uji reference string, perbandingan algoritma, serta pengujian Belady's Anomaly. Melalui simulasi ini, pengguna dapat memahami perilaku algoritma paging dan dampaknya terhadap performa sistem memori virtual.

1.3.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <time.h>
#include <limits.h>

#define FRAME_SIZE 256
#define MAX_FRAMES 16
#define PAGE_SIZE FRAME_SIZE

// Algoritma yang tersedia
typedef enum {
    FIFO,
    LRU
} Algorithm;

typedef struct {
    int page_number;
    int frame_number;
    int valid;
```

```

    int reference_bit;

    int modify_bit;

    int last_used; // Untuk LRU - timestamp penggunaan terakhir
} PageTableEntry;

typedef struct {

    int occupied;

    int page_number;

    int load_time; // Untuk FIFO - waktu pemuatan halaman

    int dirty; // Untuk statistik - halaman telah dimodifikasi
} Frame;

// Variabel global

Frame physical_memory[MAX_FRAMES];

PageTableEntry page_table[100];

int num_pages = 0;

int num_frames = 4; // Default 4 frames untuk test scenario

Algorithm current_algorithm = FIFO;

// Statistik

int page_faults = 0;

int page_hits = 0;

int disk_writes = 0;

int total_accesses = 0;

// Untuk FIFO

int fifo_queue[MAX_FRAMES];

int front = 0, rear = -1, queue_count = 0;

// Untuk LRU

int lru_counter = 0;

void initialize_system(int frames) {

    num_frames = frames;

    for (int i = 0; i < MAX_FRAMES; i++) {

        physical_memory[i].occupied = 0;

        physical_memory[i].page_number = -1;

```

```

    physical_memory[i].load_time = 0;
    physical_memory[i].dirty = 0;
}

// Reset page table
for (int i = 0; i < 100; i++) {
    page_table[i].page_number = i;
    page_table[i].frame_number = -1;
    page_table[i].valid = 0;
    page_table[i].reference_bit = 0;
    page_table[i].modify_bit = 0;
    page_table[i].last_used = -1;
}

// Reset FIFO queue
front = 0;
rear = -1;
queue_count = 0;

// Reset LRU counter
lru_counter = 0;

// Reset statistik
page_faults = 0;
page_hits = 0;
disk_writes = 0;
total_accesses = 0;
}

void display_page_table() {
    printf("\n=== PAGE TABLE ===\n");
    printf("%-12s %-15s %-10s %-12s %-12s %-12s\n",
           "Page", "Frame", "Valid", "Referenced", "Modified", "Last Used");
    printf("-----\n");
    for (int i = 0; i < num_pages; i++) {

```

```

        printf("%-12d %-15d %-10s %-12s %-12s %-12d\n",
            page_table[i].page_number,
            page_table[i].frame_number,
            page_table[i].valid ? "Yes" : "No",
            page_table[i].reference_bit ? "Yes" : "No",
            page_table[i].modify_bit ? "Yes" : "No",
            page_table[i].last_used);
    }
    printf("\n");
}

void display_physical_memory() {
    printf("=== PHYSICAL MEMORY (%d FRAMES) ===\n", num_frames);
    printf("%-10s %-10s %-15s %-10s %-10s\n",
        "Frame", "Status", "Page Number", "Load Time", "Dirty");
    printf("-----\n");
    for (int i = 0; i < num_frames; i++) {
        printf("%-10d %-10s", i, physical_memory[i].occupied ? "Occupied" : "Free");
        if (physical_memory[i].occupied) {
            printf("%-15d %-10d %-10s",
                physical_memory[i].page_number,
                physical_memory[i].load_time,
                physical_memory[i].dirty ? "Yes" : "No");
        }
        printf("\n");
    }
    printf("\n");
}

int find_free_frame() {
    for (int i = 0; i < num_frames; i++) {
        if (!physical_memory[i].occupied) {
            return i;
        }
    }
}

```

```

    }
}
return -1;
}
void enqueue_fifo(int page_num) {
    rear = (rear + 1) % num_frames;
    fifo_queue[rear] = page_num;
    if (queue_count < num_frames) {
        queue_count++;
    }
}
int dequeue_fifo() {
    if (queue_count == 0) return -1;
    int page_num = fifo_queue[front];
    front = (front + 1) % num_frames;
    queue_count--;
    return page_num;
}
int find_lru_victim() {
    int victim_frame = -1;
    int min_last_used = INT_MAX;
    for (int i = 0; i < num_frames; i++) {
        if (physical_memory[i].occupied) {
            int page_num = physical_memory[i].page_number;
            if (page_table[page_num].last_used < min_last_used) {
                min_last_used = page_table[page_num].last_used;
                victim_frame = i;
            }
        }
    }
    return victim_frame;
}

```



```

}

int find_fifo_victim() {
    // FIFO menggunakan queue, jadi kita ambil dari depan
    if (queue_count == 0) return -1;
    int victim_page = fifo_queue[front];
    for (int i = 0; i < num_frames; i++) {
        if (physical_memory[i].occupied && physical_memory[i].page_number == victim_page) {
            return i;
        }
    }
    return -1;
}

int replace_page(int page_num, int write) {
    int victim_frame = -1;
    if (current_algorithm == FIFO) {
        victim_frame = find_fifo_victim();
    } else if (current_algorithm == LRU) {
        victim_frame = find_lru_victim();
    }
    if (victim_frame == -1) {
        return -1;
    }
    int victim_page = physical_memory[victim_frame].page_number;
    // Jika halaman korban telah dimodifikasi, perlu write ke disk
    if (page_table[victim_page].modify_bit && physical_memory[victim_frame].dirty) {
        disk_writes++;
        printf(" [DISK WRITE] Page %d (dirty) ditulis ke disk\n", victim_page);
    }

    // Update page table untuk halaman korban

```

```

page_table[victim_page].valid = 0;
page_table[victim_page].frame_number = -1;
page_table[victim_page].modify_bit = 0;

// Update page table untuk halaman baru
page_table[page_num].frame_number = victim_frame;
page_table[page_num].valid = 1;
page_table[page_num].reference_bit = 1;
page_table[page_num].last_used = lru_counter++;
if (write) {
    page_table[page_num].modify_bit = 1;
}
// Update physical memory
physical_memory[victim_frame].page_number = page_num;
physical_memory[victim_frame].load_time = total_accesses;
physical_memory[victim_frame].dirty = write ? 1 : 0;

// Update FIFO queue jika menggunakan FIFO
if (current_algorithm == FIFO) {
    dequeue_fifo();
    enqueue_fifo(page_num);
}
return victim_frame;
}

int allocate_page(int page_num, int write) {
    total_accesses++;
    if (page_num >= 100) {
        printf("Error: Page number terlalu besar\n");
        return -1;
    }
    if (page_table[page_num].valid) {

```

```

page_hits++;
page_table[page_num].reference_bit = 1;
page_table[page_num].last_used = lru_counter++;
if (write) {
    page_table[page_num].modify_bit = 1;
    // Update dirty bit di physical memory
    int frame_num = page_table[page_num].frame_number;
    if (frame_num >= 0 && frame_num < num_frames) {
        physical_memory[frame_num].dirty = 1;
    }
}
return page_table[page_num].frame_number;
}
page_faults++;
int frame = find_free_frame();
if (frame == -1) {
    // Tidak ada frame kosong, perlu page replacement
    printf(" [PAGE REPLACEMENT] Menggunakan algoritma %s\n",
        current_algorithm == FIFO ? "FIFO" : "LRU");
    frame = replace_page(page_num, write);
    if (frame == -1) {
        printf("Error: Gagal melakukan page replacement\n");
        return -1;
    }
    printf(" Page %d menggantikan page di frame %d\n", page_num, frame);
} else {
    page_table[page_num].frame_number = frame;
    page_table[page_num].valid = 1;
    page_table[page_num].reference_bit = 1;
    page_table[page_num].last_used = lru_counter++;
    if (write) {

```

```

        page_table[page_num].modify_bit = 1;
    }
    physical_memory[frame].occupied = 1;
    physical_memory[frame].page_number = page_num;
    physical_memory[frame].load_time = total_accesses;
    physical_memory[frame].dirty = write ? 1 : 0;
    if (current_algorithm == FIFO) {
        enqueue_fifo(page_num);
    }

    printf("Page %d dialokasikan ke frame %d\n", page_num, frame);
}
if (page_num >= num_pages) {
    num_pages = page_num + 1;
}
return frame;
}

void display_statistics() {
    printf("\n=== STATISTIK ===\n");
    printf("Algoritma: %s\n", current_algorithm == FIFO ? "FIFO" : "LRU");
    printf("Jumlah Frames: %d\n", num_frames);
    printf("Total Akses: %d\n", total_accesses);
    printf("Page Faults: %d\n", page_faults);
    printf("Page Hits: %d\n", page_hits);

    if (total_accesses > 0) {
        float fault_rate = (float)page_faults / total_accesses * 100;
        float hit_rate = (float)page_hits / total_accesses * 100;
        printf("Page Fault Rate: %.2f%%\n", fault_rate);
        printf("Hit Rate: %.2f%%\n", hit_rate);
    }
}

```

```
printf("Disk Writes: %d\n", disk_writes);

printf("\n");
}

void run_test_scenario(int frames, Algorithm algo) {

printf("\n\n=====
===== \n");

printf("|| TEST SCENARIO - %s dengan %d frames ||\n",
algo == FIFO ? "FIFO" : "LRU", frames);

printf("L
=====
===== \n\n");

initialize_system(frames);

current_algorithm = algo;

int reference_string[] = {7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};

int length = sizeof(reference_string) / sizeof(reference_string[0]);

printf("Step\tReference\tFrame 0\tFrame 1\tFrame 2\tFrame 3\tFault?\n");

printf("-----\n");

for (int i = 0; i < length; i++) {

int page_num = reference_string[i];

int fault_before = page_faults;

allocate_page(page_num, 0);

printf("%d\t%d\t\t", i+1, page_num);

for (int f = 0; f < frames; f++) {

if (physical_memory[f].occupied) {

printf("%d\t", physical_memory[f].page_number);

} else {
```

```

        printf("-\t");
    }
}

if (page_faults > fault_before) {
    printf("FAULT");
} else {
    printf("HIT");
}

printf("\n");
}

display_statistics();
}

void compare_algorithms() {

printf("\n\n===== \n");

printf("|| PERBANDINGAN ALGORITMA (4 frames) ||\n");

printf("||===== \n\n");

printf("%-15s %-20s %-15s %-20s\n",
       "Algoritma", "Jumlah Page Fault", "Hit Ratio", "Disk Writes");
printf("-----\n");

// Jalankan FIFO
run_test_scenario(4, FIFO);

int fifo_faults = page_faults;
int fifo_writes = disk_writes;

float fifo_hit_rate = (float)page_hits / total_accesses * 100;

// Jalankan LRU
run_test_scenario(4, LRU);

int lru_faults = page_faults;
```

```

int lru_writes = disk_writes;

float lru_hit_rate = (float)page_hits / total_accesses * 100;

printf("\n=== TABEL PERBANDINGAN ===\n");

printf("%-15s %-20d %-15.2f%% %-20d\n",
        "FIFO", fifo_faults, fifo_hit_rate, fifo_writes);

printf("%-15s %-20d %-15.2f%% %-20d\n",
        "LRU", lru_faults, lru_hit_rate, lru_writes);
}

void belady_anomaly_test() {

printf("\n=====
===== \n");

printf("|| BELADY'S ANOMALY TEST || \n");

printf("||=====
===== \n\n");

printf("Menguji apakah page fault berkurang seiring bertambahnya frames:\n\n");

// Test untuk 3, 4, dan 5 frames
int frames_list[] = {3, 4, 5};

int num_tests = sizeof(frames_list) / sizeof(frames_list[0]);

printf("%-10s %-15s %-15s\n", "Frames", "FIFO Faults", "LRU Faults");
printf("-----\n");

for (int i = 0; i < num_tests; i++) {
    int frames = frames_list[i];

    // Test FIFO
    run_test_scenario(frames, FIFO);

    int fifo_faults = page_faults;

    // Test LRU
    run_test_scenario(frames, LRU);

    int lru_faults = page_faults;

    printf("%-10d %-15d %-15d\n", frames, fifo_faults, lru_faults);
}

```

```

}

printf("\nANALISIS:\n");

printf("Belady's Anomaly terjadi ketika jumlah page fault BERTAMBAH\n");

printf("meskipun jumlah frame ditambah. Fenomena ini bisa terjadi\n");

printf("pada algoritma FIFO tetapi tidak pada LRU.\n");
}

```

```

void demo_scenario() {

```

```

    printf("\n=====
===== \n");

```

```

    printf("|| DEMO SCENARIO: PAGING SYSTEM ||\n");

```

```

    printf("=====
===== \n\n");

```

```

    initialize_system(4);

```

```

    current_algorithm = FIFO;

```

```

    printf("Skenario: Program berukuran 2048 bytes\n");

```

```

    printf("Jumlah pages: %d pages\n\n", 2048 / PAGE_SIZE);

```

```

    // Simulasi beberapa akses

```

```

    int accesses[][2] = {

```

```

        {0, 0}, // Read address 0

```

```

        {300, 1}, // Write address 300

```

```

        {512, 0}, // Read address 512

```

```

        {100, 0}, // Read address 100

```

```

        {1024, 1} // Write address 1024

```

```

    };

```

```

    for (int i = 0; i < 5; i++) {

```

```

        int addr = accesses[i][0];

```

```

        int write = accesses[i][1];

```

```

        printf("\nAkses %s pada logical address %d:\n",

```

```

            write ? "WRITE" : "READ", addr);

```

```

        int page_num = addr / PAGE_SIZE;

```



```

    int frame = allocate_page(page_num, write);

    if (frame != -1) {
        printf(" Page %d -> Frame %d\n", page_num, frame);
    }
}

display_page_table();
display_physical_memory();
display_statistics();
}

int main() {
    int choice, address, write, frames, algo_choice;

    // Default initialization
    initialize_system(4);

    while(1) {
        printf("\n=== ADVANCED PAGING SIMULATOR ===\n");
        printf("1. Set Jumlah Frames\n");
        printf("2. Set Algoritma (FIFO/LRU)\n");
        printf("3. Allocate Page\n");
        printf("4. Access Memory (Read)\n");
        printf("5. Access Memory (Write)\n");
        printf("6. Display Page Table\n");
        printf("7. Display Physical Memory\n");
        printf("8. Display Statistics\n");
        printf("9. Run Demo Scenario\n");
        printf("10. Run Test Scenario (Reference String)\n");
        printf("11. Compare FIFO vs LRU\n");
        printf("12. Belady's Anomaly Test\n");
        printf("13. Reset System\n");
        printf("14. Exit\n");
        printf("Pilihan: ");

```

```
scanf("%d", &choice);

switch(choice) {

    case 1:

        printf("Masukkan jumlah frames (1-%d): ", MAX_FRAMES);

        scanf("%d", &frames);

        if (frames > 0 && frames <= MAX_FRAMES) {

            initialize_system(frames);

            printf("Sistem direset dengan %d frames\n", frames);

        } else {

            printf("Jumlah frames tidak valid!\n");

        }

        break;

    case 2:

        printf("Pilih algoritma:\n");

        printf("1. FIFO\n");

        printf("2. LRU\n");

        printf("Pilihan: ");

        scanf("%d", &algo_choice);

        if (algo_choice == 1) {

            current_algorithm = FIFO;

            printf("Algoritma diubah menjadi FIFO\n");

        } else if (algo_choice == 2) {

            current_algorithm = LRU;

            printf("Algoritma diubah menjadi LRU\n");

        } else {

            printf("Pilihan tidak valid!\n");

        }

        break;

    case 3:

        printf("Masukkan page number: ");

        scanf("%d", &address);
```

```
printf("Tipe akses (0=Read, 1=Write): ");
```

```
scanf("%d", &write);
```

```
allocate_page(address, write);
```

```
break;
```

case 4:

```
printf("Masukkan logical address: ");
```

```
scanf("%d", &address);
```

```
printf("Page %d diakses (READ)\n", address / PAGE_SIZE);
```

```
allocate_page(address / PAGE_SIZE, 0);
```

```
break;
```

case 5:

```
printf("Masukkan logical address: ");
```

```
scanf("%d", &address);
```

```
printf("Page %d diakses (WRITE)\n", address / PAGE_SIZE);
```

```
allocate_page(address / PAGE_SIZE, 1);
```

```
break;
```

case 6:

```
display_page_table();
```

```
break;
```

case 7:

```
display_physical_memory();
```

```
break;
```

case 8:

```
display_statistics();
```

```
break;
```

case 9:

```
demo_scenario();
```

```
break;
```

case 10:

```
printf("Masukkan jumlah frames untuk test: ");
```

```

scanf("%d", &frames);

printf("Pilih algoritma (1=FIFO, 2=LRU): ");

scanf("%d", &algo_choice);

if (frames > 0 && frames <= MAX_FRAMES && (algo_choice == 1 || algo_choice == 2)) {
    run_test_scenario(frames, algo_choice == 1 ? FIFO : LRU);
} else {
    printf("Input tidak valid!\n");
}

break;
case 11:
    compare_algorithms();
    break;
case 12:
    belady_anomaly_test();
    break;
case 13:
    initialize_system(num_frames);
    printf("Sistem direset\n");
    break;
case 14:
    printf("Keluar dari program...\n");
    return 0;
default:
    printf("Pilihan tidak valid!\n");
}
}

return 0;
}

```

1.3.3 Screenshots Output

```

gufro@MSI:~/praktikum-memory/architecture$ nano paging_simulator.c
gufro@MSI:~/praktikum-memory/architecture$ gcc -o paging_simulator paging_simulator.c
gufro@MSI:~/praktikum-memory/architecture$ ./paging_simulator

```

```

=== ADVANCED PAGING SIMULATOR ===
1. Set Jumlah Frames
2. Set Algoritma (FIFO/LRU)
3. Allocate Page
4. Access Memory (Read)
5. Access Memory (Write)
6. Display Page Table
7. Display Physical Memory
8. Display Statistics
9. Run Demo Scenario
10. Run Test Scenario (Reference String)
11. Compare FIFO vs LRU
12. Belady's Anomaly Test
13. Reset System
14. Exit
Pilihan: 10
Masukkan jumlah frames untuk test: 10
Pilih algoritma (1=FIFO, 2=LRU): 2

```

TEST SCENARIO - LRU dengan 10 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
Page 7	dialokasikan ke frame 0	7	-	-	-	-
Page 0	dialokasikan ke frame 1	0	7	-	-	-
Page 1	dialokasikan ke frame 2	1	0	7	-	-
Page 2	dialokasikan ke frame 3	2	0	1	7	-
Page 3	dialokasikan ke frame 4	3	0	1	2	7
Page 4	dialokasikan ke frame 5	4	0	1	2	3
Page 5	dialokasikan ke frame 6	5	0	1	2	3
Page 6	dialokasikan ke frame 7	6	0	1	2	3
Page 7	dialokasikan ke frame 8	7	0	1	2	3
Page 8	dialokasikan ke frame 9	8	0	1	2	3
Page 9	dialokasikan ke frame 10	9	0	1	2	3
Page 10	dialokasikan ke frame 11	10	0	1	2	3
Page 11	dialokasikan ke frame 12	11	0	1	2	3
Page 12	dialokasikan ke frame 13	12	0	1	2	3
Page 13	dialokasikan ke frame 14	13	0	1	2	3
Page 14	dialokasikan ke frame 15	14	0	1	2	3
Page 15	dialokasikan ke frame 16	15	0	1	2	3
Page 16	dialokasikan ke frame 17	16	0	1	2	3
Page 17	dialokasikan ke frame 18	17	0	1	2	3
Page 18	dialokasikan ke frame 19	18	0	1	2	3
Page 19	dialokasikan ke frame 20	19	0	1	2	3
Page 20	dialokasikan ke frame 21	20	0	1	2	3

```

=== STATISTIK ===
Algoritma: LRU
Jumlah Frames: 10
Total Akses: 20
Page Faults: 6
Page Hits: 14
Page Fault Rate: 30.00%
Hit Rate: 70.00%
Disk Writes: 0

```

```

=== ADVANCED PAGING SIMULATOR ===
1. Set Jumlah Frames
2. Set Algoritma (FIFO/LRU)
3. Allocate Page
4. Access Memory (Read)
5. Access Memory (Write)
6. Display Page Table
7. Display Physical Memory
8. Display Statistics
9. Run Demo Scenario
10. Run Test Scenario (Reference String)
11. Compare FIFO vs LRU
12. Belady's Anomaly Test
13. Reset System
14. Exit
Pilihan: 11

```

PERBANDINGAN ALGORITMA (4 frames)

Algoritma	Jumlah Page Fault	Hit Ratio	Disk Writes
-----------	-------------------	-----------	-------------

TEST SCENARIO - FIFO dengan 4 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?		
Page 7	dialokasikan ke frame 0	7	-	-	-	FAULT		
Page 0	dialokasikan ke frame 1	0	7	-	-	FAULT		
Page 1	dialokasikan ke frame 2	1	0	-	-	FAULT		
Page 2	dialokasikan ke frame 3	2	0	1	-	FAULT		
Page 3	dialokasikan ke frame 4	3	0	1	2	HIT		
Page 4	dialokasikan ke frame 5	4	0	1	2	HIT		
[PAGE REPLACEMENT] Menggunakan algoritma FIFO								
Page 5	menggantikan page di frame 0	5	0	1	2	FAULT		
Page 6	menggantikan page di frame 1	6	3	0	1	2	HIT	
[PAGE REPLACEMENT] Menggunakan algoritma FIFO								
Page 4	menggantikan page di frame 1	4	3	0	1	2	FAULT	
Page 8	menggantikan page di frame 2	8	3	4	1	2	HIT	
Page 9	menggantikan page di frame 3	9	3	4	1	2	HIT	
[PAGE REPLACEMENT] Menggunakan algoritma FIFO								
Page 0	menggantikan page di frame 2	0	3	4	0	2	FAULT	
Page 11	menggantikan page di frame 3	11	3	4	0	2	HIT	
Page 12	menggantikan page di frame 4	12	3	4	0	2	HIT	
Page 13	menggantikan page di frame 5	13	2	3	4	0	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO								
Page 1	menggantikan page di frame 3	1	4	0	1	2	FAULT	
[PAGE REPLACEMENT] Menggunakan algoritma FIFO								
Page 2	menggantikan page di frame 0	2	4	0	1	2	FAULT	
Page 16	menggantikan page di frame 1	16	0	2	4	0	1	HIT
Page 17	menggantikan page di frame 0	17	1	2	4	0	1	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO								
Page 7	menggantikan page di frame 1	7	2	7	0	1	2	FAULT
Page 18	menggantikan page di frame 2	18	2	7	0	1	2	HIT
Page 19	menggantikan page di frame 3	19	2	7	0	1	2	HIT
Page 20	menggantikan page di frame 4	20	1	2	7	0	1	HIT

```

=== STATISTIK ===
Algoritma: FIFO
Jumlah Frames: 4
Total Akses: 20
Page Faults: 19
Page Hits: 1
Page Fault Rate: 95.00%
Hit Rate: 5.00%
Disk Writes: 0

```

```
TEST SCENARIO - LRU dengan 4 frames

Step  Reference      Frame 0 Frame 1 Frame 2 Frame 3 Fault?
-----
Page 7 dialokasikan ke frame 0
1      7              7      -      -      -      FAULT
Page 0 dialokasikan ke frame 1
2      0              7      0      -      -      FAULT
Page 1 dialokasikan ke frame 2
3      1              7      0      1      -      FAULT
Page 2 dialokasikan ke frame 3
4      2              7      0      1      2      FAULT
5      0              7      0      1      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 3 menggantikan page di frame 0
6      3              3      0      1      2      FAULT
7      0              3      0      1      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 4 menggantikan page di frame 2
8      4              3      0      4      2      FAULT
9      2              3      0      4      2      HIT
10     3              3      0      4      2      HIT
11     0              3      0      4      2      HIT
12     3              3      0      4      2      HIT
13     2              3      0      4      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 1 menggantikan page di frame 2
14     1              3      0      1      2      FAULT
15     2              3      0      1      2      HIT
16     0              3      0      1      2      HIT
17     1              3      0      1      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 7 menggantikan page di frame 0
18     7              7      0      1      2      FAULT
19     0              7      0      1      2      HIT
20     1              7      0      1      2      HIT

=== STATISTIK ===
Algoritma: LRU
Jumlah Frames: 4
Total Akses: 20
Page Faults: 8
Page Hits: 12
Page Fault Rate: 40.00%
Hit Rate: 60.00%
Disk Writes: 0
```

```
=== TABEL PERBANDINGAN ===
FIFO      10      50.00      % 0
LRU       8       60.00      % 0

=== ADVANCED PAGING SIMULATOR ===
1. Set Jumlah Frames
2. Set Algoritma (FIFO/LRU)
3. Allocate Page
4. Access Memory (Read)
5. Access Memory (Write)
6. Display Page Table
7. Display Physical Memory
8. Display Statistics
9. Run Demo Scenario
10. Run Test Scenario (Reference String)
11. Compare FIFO vs LRU
12. Belady's Anomaly Test
13. Reset System
14. Exit
Pilihan: 12

BELADY'S ANOMALY TEST

Menguji apakah page fault berkurang seiring bertambahnya frames:

Frames      FIFO Faults      LRU Faults
-----
TEST SCENARIO - FIFO dengan 3 frames

Step  Reference      Frame 0 Frame 1 Frame 2 Frame 3 Fault?
-----
Page 7 dialokasikan ke frame 0
1      7              7      -      -      -      FAULT
Page 0 dialokasikan ke frame 1
2      0              7      0      -      -      FAULT
Page 1 dialokasikan ke frame 2
3      1              7      0      1      -      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 2 menggantikan page di frame 0
4      2              2      0      1      -      FAULT
5      0              2      0      1      -      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 3 menggantikan page di frame 1
6      3              2      3      1      -      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 0 menggantikan page di frame 2
7      0              2      3      0      -      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 4 menggantikan page di frame 0
8      4              4      3      0      -      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
```

```

8      4      0      4      3      0      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 2 menggantikan page di frame 1
9      2      4      2      0      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 3 menggantikan page di frame 2
10     3      4      2      3      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 0 menggantikan page di frame 0
11     0      0      2      3      FAULT
12     3      0      2      3      HIT
13     2      0      2      3      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 1 menggantikan page di frame 1
14     1      0      1      3      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 2 menggantikan page di frame 2
15     2      0      1      2      FAULT
16     0      0      1      2      HIT
17     1      0      1      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 7 menggantikan page di frame 0
18     7      7      1      2      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 0 menggantikan page di frame 1
19     0      7      0      2      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO
Page 1 menggantikan page di frame 2
20     1      7      0      1      FAULT

=== STATISTIK ===
Algoritma: FIFO
Jumlah Frames: 3
Total Akses: 20
Page Faults: 15
Page Hits: 5
Page Fault Rate: 75.00%
Hit Rate: 25.00%
Disk Writes: 0

```

```

TEST SCENARIO - LRU dengan 3 frames

Step  Reference      Frame 0 Frame 1 Frame 2 Frame 3 Fault?
-----
Page 7 dialokasikan ke frame 0
1      7      7      -      -      FAULT
Page 0 dialokasikan ke frame 1
2      0      7      0      -      FAULT
Page 1 dialokasikan ke frame 2
3      1      7      0      1      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 2 menggantikan page di frame 0
4      2      2      0      1      FAULT
5      0      2      0      1      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 3 menggantikan page di frame 2
6      3      2      0      3      FAULT
7      0      2      0      3      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 4 menggantikan page di frame 0
8      4      4      0      3      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 2 menggantikan page di frame 2
9      2      4      0      2      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 3 menggantikan page di frame 1
10     3      4      3      2      FAULT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 0 menggantikan page di frame 0
11     0      0      3      2      FAULT
12     3      0      3      2      HIT
13     2      0      3      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 1 menggantikan page di frame 0
14     1      1      3      2      FAULT
15     2      1      3      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 0 menggantikan page di frame 1
16     0      1      0      2      FAULT
17     1      1      0      2      HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU
Page 7 menggantikan page di frame 2
18     7      1      0      7      FAULT
19     0      1      0      7      HIT
20     1      1      0      7      HIT

=== STATISTIK ===
Algoritma: LRU
Jumlah Frames: 3
Total Akses: 20
Page Faults: 12
Page Hits: 8
Page Fault Rate: 60.00%
Hit Rate: 40.00%
Disk Writes: 0

```

TEST SCENARIO - FIFO dengan 4 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
Page 7	dialokasikan ke frame 0					
1	7	7	-	-	-	FAULT
Page 0	dialokasikan ke frame 1					
2	0	7	0	-	-	FAULT
Page 1	dialokasikan ke frame 2					
3	1	7	0	1	-	FAULT
Page 2	dialokasikan ke frame 3					
4	2	7	0	1	2	FAULT
5	0	7	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 3	menggantikan page di frame 0					
6	3	3	0	1	2	FAULT
7	0	3	0	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 4	menggantikan page di frame 1					
8	4	3	4	1	2	FAULT
9	2	3	4	1	2	HIT
10	3	3	4	1	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 0	menggantikan page di frame 2					
11	0	3	4	0	2	FAULT
12	3	3	4	0	2	HIT
13	2	3	4	0	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 1	menggantikan page di frame 3					
14	1	3	4	0	1	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 2	menggantikan page di frame 0					
15	2	2	4	0	1	FAULT
16	0	2	4	0	1	HIT
17	1	2	4	0	1	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO						
Page 7	menggantikan page di frame 1					
18	7	2	7	0	1	FAULT
19	0	2	7	0	1	HIT
20	1	2	7	0	1	HIT

```

=== STATISTIK ===
Algoritma: FIFO
Jumlah Frames: 4
Total Akses: 20
Page Faults: 10
Page Hits: 10
Page Fault Rate: 50.00%
Hit Rate: 50.00%
Disk Writes: 0

```

TEST SCENARIO - LRU dengan 4 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
Page 7	diaksesikan ke frame 1	1	-	-	-	FAULT
Page 0	diaksesikan ke frame 0	0	-	-	-	
Page 1	diaksesikan ke frame 2	0	1	-	-	FAULT
Page 3	diaksesikan ke frame 0	0	1	1	-	FAULT
Page 2	diaksesikan ke frame 3	0	1	-	1	FAULT
Page 5	diaksesikan ke frame 0	0	1	2	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 3	menggantikan page di frame 0	0	1	2	2	FAULT
Page 7	diaksesikan ke frame 1	0	1	2	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 4	menggantikan page di frame 2	0	1	2	2	FAULT
Page 8	diaksesikan ke frame 0	0	4	2	2	FAULT
Page 9	diaksesikan ke frame 3	0	4	2	2	HIT
Page 10	diaksesikan ke frame 0	0	4	2	2	HIT
Page 11	diaksesikan ke frame 4	0	4	2	2	HIT
Page 12	diaksesikan ke frame 3	0	4	2	2	HIT
Page 13	diaksesikan ke frame 0	0	4	2	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 1	menggantikan page di frame 2	0	1	2	2	FAULT
Page 14	diaksesikan ke frame 0	0	1	2	2	FAULT
Page 15	diaksesikan ke frame 1	0	1	2	2	HIT
Page 16	diaksesikan ke frame 0	0	1	2	2	HIT
Page 17	diaksesikan ke frame 1	0	1	2	2	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU						
Page 7	menggantikan page di frame 0	0	1	2	2	FAULT
Page 18	diaksesikan ke frame 0	0	1	2	2	HIT
Page 19	diaksesikan ke frame 1	0	1	2	2	HIT
Page 20	diaksesikan ke frame 0	0	1	2	2	HIT

```
=== STATISTIK ===
Algoritma: LRU
Jumlah Frames: 4
Total Akses: 20
Page Faults: 8
Page Hits: 12
Page Fault Rate: 40.00%
Hit Rate: 60.00%
Disk Writes: 0
```

TEST SCENARIO – FIFO dengan 5 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
Page 7 dialokasikan ke frame 0						

TEST SCENARIO - FIFO dengan 5 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?	
Page 7 dialokasikan ke frame 0							
1	7	7	-	-	-	FAULT	
Page 0 dialokasikan ke frame 1							
2	0	7	0	-	-	FAULT	
Page 1 dialokasikan ke frame 2							
3	1	7	0	1	-	FAULT	
Page 2 dialokasikan ke frame 3							
4	2	7	0	1	2	FAULT	
5	0	7	0	1	2	HIT	
Page 3 dialokasikan ke frame 4							
6	3	7	0	1	2	3	FAULT
7	0	7	0	1	2	3	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO							
Page 4 menggantikan page di frame 0							
8	4	4	0	1	2	3	FAULT
9	2	4	0	1	2	3	HIT
10	3	4	0	1	2	3	HIT
11	0	4	0	1	2	3	HIT
12	3	4	0	1	2	3	HIT
13	2	4	0	1	2	3	HIT
14	1	4	0	1	2	3	HIT
15	2	4	0	1	2	3	HIT
16	0	4	0	1	2	3	HIT
17	1	4	0	1	2	3	HIT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO							
Page 7 menggantikan page di frame 1							
18	7	4	7	1	2	3	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO							
Page 0 menggantikan page di frame 2							
19	0	4	7	0	2	3	FAULT
[PAGE REPLACEMENT] Menggunakan algoritma FIFO							
Page 1 menggantikan page di frame 3							
20	1	4	7	0	1	3	FAULT

=== STATISTIK ===

Algoritma: FIFO
Jumlah Frames: 5
Total Akses: 20
Page Faults: 9
Page Hits: 11
Page Fault Rate: 45.00%
Hit Rate: 55.00%
Disk Writes: 0

=== STATISTIK ===
 Algoritma: FIFO
 Jumlah Frames: 5
 Total Akses: 20
 Page Faults: 9
 Page Hits: 11
 Page Fault Rate: 45.00%
 Hit Rate: 55.00%
 Disk Writes: 0

TEST SCENARIO - LRU dengan 5 frames

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?	
Page 7 dialokasikan ke frame 0							
1	7	7	-	-	-	FAULT	
Page 0 dialokasikan ke frame 1							
2	0	7	0	-	-	FAULT	
Page 1 dialokasikan ke frame 2							
3	1	7	0	1	-	FAULT	
Page 2 dialokasikan ke frame 3							
4	2	7	0	1	2	FAULT	
5	0	7	0	1	2	HIT	
Page 3 dialokasikan ke frame 4							
6	3	7	0	1	2	3	FAULT
7	0	7	0	1	2	3	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU							
Page 4 menggantikan page di frame 0							
8	4	4	0	1	2	3	FAULT
9	2	4	0	1	2	3	HIT
10	3	4	0	1	2	3	HIT
11	0	4	0	1	2	3	HIT
12	3	4	0	1	2	3	HIT
13	2	4	0	1	2	3	HIT
14	1	4	0	1	2	3	HIT
15	2	4	0	1	2	3	HIT
16	0	4	0	1	2	3	HIT
17	1	4	0	1	2	3	HIT
[PAGE REPLACEMENT] Menggunakan algoritma LRU							
Page 7 menggantikan page di frame 0							
18	7	7	0	1	2	3	FAULT
19	0	7	0	1	2	3	HIT
20	1	7	0	1	2	3	HIT

=== STATISTIK ===
 Algoritma: LRU
 Jumlah Frames: 5
 Total Akses: 20
 Page Faults: 7
 Page Hits: 13
 Page Fault Rate: 35.00%
 Hit Rate: 65.00%
 Disk Writes: 0

5 9 7

ANALISIS:
 Belady's Anomaly terjadi ketika jumlah page fault BERTAMBAH meskipun jumlah frame ditambah. Fenomena ini bisa terjadi pada algoritma FIFO tetapi tidak pada LRU.

=== ADVANCED PAGING SIMULATOR ===

1. Set Jumlah Frames
2. Set Algoritma (FIFO/LRU)
3. Allocate Page
4. Access Memory (Read)
5. Access Memory (Write)
6. Display Page Table
7. Display Physical Memory

1.3.4 Emplementasi

Program ini mengimplementasikan simulator sistem paging lanjutan yang mendukung algoritma FIFO dan LRU untuk page replacement. Sistem memodelkan page table, physical memory, serta statistik seperti page fault, page hit, dan disk write. Setiap akses memori akan dicek apakah halaman sudah berada di frame (hit) atau menyebabkan page fault. Jika memori penuh, algoritma yang dipilih menentukan halaman korban untuk digantikan. FIFO menggunakan antrian kedatangan halaman, sedangkan LRU menggunakan timestamp pemakaian terakhir. Program menyediakan menu interaktif, skenario uji reference string, perbandingan algoritma, serta pengujian Belady's Anomaly untuk menganalisis kinerja manajemen memori virtual.

1.4 TOPIK 4 Multi-Level Page Table

1.4.1 Deskripsi Tugas

2. Tugas ini bertujuan untuk mensimulasikan mekanisme manajemen memori virtual menggunakan struktur three-level page table dan inverted page table. Program dirancang untuk menunjukkan proses pembagian alamat virtual menjadi beberapa level indeks, alokasi page secara dinamis, serta translasi alamat virtual ke alamat fisik. Selain itu, tugas ini menganalisis memory overhead pada three-level page table dengan menghitung jumlah tabel yang aktif. Implementasi inverted page table digunakan untuk membandingkan efisiensi memori dengan pendekatan tabel global berbasis frame fisik.

2.1.1 Source Code

```
#include <stdio.h>

#include <stdlib.h>

#define L1_BITS 16
#define L2_BITS 8
#define L3_BITS 8
#define OFFSET_BITS 12

#define NUM_FRAMES 1024 // untuk inverted page table

/* ===== */
/* THREE-LEVEL PAGE TABLE */
/* ===== */
```

```

typedef struct {
    int frame_number;
    int valid;
    int read, write, execute;
} L3_Entry;

typedef struct {
    L3_Entry *entries;
    int valid;
} L2_Entry;

typedef struct {
    L2_Entry *entries;
    int valid;
} L1_Entry;

L1_Entry *L1;

/* Ambil index dari virtual address */
unsigned int get_l1(unsigned long va) {
    return (va >> (L2_BITS + L3_BITS + OFFSET_BITS)) & ((1 << L1_BITS) - 1);
}

unsigned int get_l2(unsigned long va) {
    return (va >> (L3_BITS + OFFSET_BITS)) & ((1 << L2_BITS) - 1);
}

unsigned int get_l3(unsigned long va) {
    return (va >> OFFSET_BITS) & ((1 << L3_BITS) - 1);
}

```

```

/* Alokasi page */

void allocate_3level(unsigned long va, int frame) {

    int l1 = get_l1(va);
    int l2 = get_l2(va);
    int l3 = get_l3(va);

    if (!L1[l1].valid) {
        L1[l1].entries = calloc(1 << L2_BITS, sizeof(L2_Entry));
        L1[l1].valid = 1;
    }

    if (!L1[l1].entries[l2].valid) {
        L1[l1].entries[l2].entries = calloc(1 << L3_BITS, sizeof(L3_Entry));
        L1[l1].entries[l2].valid = 1;
    }

    L1[l1].entries[l2].entries[l3].frame_number = frame;
    L1[l1].entries[l2].entries[l3].valid = 1;
}

/* Translasi alamat */

int translate_3level(unsigned long va) {

    int l1 = get_l1(va);
    int l2 = get_l2(va);
    int l3 = get_l3(va);
    int offset = va & ((1 << OFFSET_BITS) - 1);

    if (!L1[l1].valid) return -1;
    if (!L1[l1].entries[l2].valid) return -1;
    if (!L1[l1].entries[l2].entries[l3].valid) return -1;

```

```

    return (L1[l1].entries[l2].entries[l3].frame_number << OFFSET_BITS) | offset;
}

/* Hitung memory overhead */
void calculate_memory_overhead() {
    int l1_count = 1 << L1_BITS;
    int l2_count = 0, l3_count = 0;

    for (int i = 0; i < l1_count; i++) {
        if (L1[i].valid) {
            l2_count++;
            for (int j = 0; j < (1 << L2_BITS); j++) {
                if (L1[i].entries[j].valid) {
                    l3_count++;
                }
            }
        }
    }
}

printf("\n=== MEMORY OVERHEAD THREE-LEVEL ===\n");
printf("L1 tables : %d\n", l1_count);
printf("L2 tables aktif : %d\n", l2_count);
printf("L3 tables aktif : %d\n", l3_count);
}

/* ===== */
/* INVERTED PAGE TABLE */
/* ===== */

typedef struct {
    int process_id;

```

```

    int page_number;

    int valid;
} InvertedPageEntry;

InvertedPageEntry inverted_table[NUM_FRAMES];

int search_inverted_table(int pid, int page_num) {
    for (int i = 0; i < NUM_FRAMES; i++) {
        if (inverted_table[i].valid &&
            inverted_table[i].process_id == pid &&
            inverted_table[i].page_number == page_num) {
            return i;
        }
    }
    return -1;
}

void insert_inverted_table(int pid, int page_num, int frame) {
    inverted_table[frame].process_id = pid;
    inverted_table[frame].page_number = page_num;
    inverted_table[frame].valid = 1;
}

/* ===== */
/* MAIN PROGRAM      */
/* ===== */

int main() {
    unsigned long va1 = 0x123456789ABC;
    unsigned long va2 = 0xABCDEF123456;

```

```

/* Inisialisasi L1 */
L1 = calloc(1 << L1_BITS, sizeof(L1_Entry));

/* Alokasi page */
allocate_3level(va1, 10);
allocate_3level(va2, 25);

/* Translasi alamat */
int pa1 = translate_3level(va1);
int pa2 = translate_3level(va2);

printf("Virtual Address 1 : %lx -> Physical Address : %x\n", va1, pa1);
printf("Virtual Address 2 : %lx -> Physical Address : %x\n", va2, pa2);

calculate_memory_overhead();

/* Inverted Page Table */
insert_inverted_table(1, 5, 100);
insert_inverted_table(2, 8, 200);

printf("\n=== INVERTED PAGE TABLE ===\n");
printf("PID 1 Page 5 -> Frame %d\n",
    search_inverted_table(1, 5));
printf("PID 2 Page 8 -> Frame %d\n",
    search_inverted_table(2, 8));

return 0;
}

```

2.1.2 Screenshots Output

```

gufro@MSI:~/praktikum-memory/architecture$ nano page_table.c
gufro@MSI:~/praktikum-memory/architecture$ gcc -o page_table page_table.c
gufro@MSI:~/praktikum-memory/architecture$ ./page_table
Virtual Address 1 : 123456789abc -> Physical Address : aabc
Virtual Address 2 : abcdef123456 -> Physical Address : 19456

=== MEMORY OVERHEAD THREE-LEVEL ===
L1 tables : 65536
L2 tables aktif : 2
L3 tables aktif : 2

=== INVERTED PAGE TABLE ===
PID 1 Page 5 -> Frame 100
PID 2 Page 8 -> Frame 200

```

2.1.3 Emplementasi

Program ini mengimplementasikan simulasi manajemen memori virtual menggunakan three-level page table dan inverted page table. Alamat virtual dibagi menjadi indeks L1, L2, L3, dan offset untuk mendukung translasi bertingkat. Tabel halaman dialokasikan secara dinamis hanya saat dibutuhkan, sehingga lebih efisien terhadap penggunaan memori. Fungsi translasi memeriksa validitas setiap level sebelum menghasilkan alamat fisik. Program juga menghitung overhead memori berdasarkan jumlah tabel aktif. Selain itu, inverted page table digunakan untuk memetakan pasangan PID dan page number langsung ke frame fisik. Implementasi ini menunjukkan perbandingan struktur tabel halaman konvensional dan inverted.

2.2 TUGAS 5 Swapping Mechanism

2.2.1 Deskripsi Tugas

3. Tugas ini bertujuan untuk mensimulasikan mekanisme swapping pada sistem operasi dengan membandingkan beberapa strategi pemilihan proses korban, yaitu FIFO (First In First Out), LRU (Least Recently Used), dan Priority-based swapping. Program mensimulasikan kondisi keterbatasan memori fisik, sehingga sebagian proses harus dipindahkan ke swap space ketika memori penuh. Setiap strategi diuji berdasarkan jumlah swap in, swap out, total waktu swapping, dan rata-rata waktu respons. Melalui simulasi ini, mahasiswa dapat memahami bagaimana kebijakan swapping memengaruhi kinerja sistem, efisiensi penggunaan memori, serta dampak strategi pemilihan korban terhadap performa sistem secara keseluruhan.

3.1.1 Source Code

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

```



```
#define MAX_PROCESSES 10

#define PHYSICAL_MEMORY 1024 // MB

#define SWAP_SPACE 2048 // MB

#define ACCESS_TIME 100

typedef enum { IN_MEMORY, SWAPPED } State;

typedef struct {
    int pid;
    int size;
    int priority;
    State state;
    time_t load_time;
    time_t last_access;
    int swap_in_count;
    int swap_out_count;
} Process;

Process processes[MAX_PROCESSES];

int used_memory = 0;
int used_swap = 0;

/* ===== METRICS ===== */

int total_swap_in = 0;
int total_swap_out = 0;
long total_swap_time = 0;

/* ===== INITIALIZATION ===== */

void init_processes() {
    srand(time(NULL));
```

```

for (int i = 0; i < MAX_PROCESSES; i++) {
    processes[i].pid = i + 1;
    processes[i].size = 50 + rand() % 151; // 50–200 MB
    processes[i].priority = 1 + rand() % 10;
    processes[i].state = SWAPPED;
    processes[i].swap_in_count = 0;
    processes[i].swap_out_count = 0;
}
}

/* ===== VICTIM SELECTION ===== */

int select_fifo() {
    time_t oldest = time(NULL);
    int victim = -1;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (processes[i].state == IN_MEMORY &&
            processes[i].load_time < oldest) {
            oldest = processes[i].load_time;
            victim = i;
        }
    }
    return victim;
}

int select_lru() {
    time_t oldest = time(NULL);
    int victim = -1;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (processes[i].state == IN_MEMORY &&
            processes[i].last_access < oldest) {

```

```

        oldest = processes[i].last_access;
        victim = i;
    }
}
return victim;
}

int select_priority() {
    int lowest = 11;
    int victim = -1;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (processes[i].state == IN_MEMORY &&
            processes[i].priority < lowest) {
            lowest = processes[i].priority;
            victim = i;
        }
    }
    return victim;
}

/* ===== SWAP OPERATIONS ===== */
void swap_out(int idx) {
    if (idx < 0) return;

    usleep(processes[idx].size * 1000);
    total_swap_time += processes[idx].size;

    used_memory -= processes[idx].size;
    used_swap += processes[idx].size;
}

```

```

    processes[idx].state = SWAPPED;
    processes[idx].swap_out_count++;
    total_swap_out++;
}

void swap_in(int idx) {
    usleep(processes[idx].size * 1000);
    total_swap_time += processes[idx].size;

    used_memory += processes[idx].size;
    used_swap -= processes[idx].size;

    processes[idx].state = IN_MEMORY;
    processes[idx].load_time = time(NULL);
    processes[idx].last_access = time(NULL);
    processes[idx].swap_in_count++;
    total_swap_in++;
}

/* ===== LOAD PROCESS ===== */
void load_process(int idx, int strategy) {
    if (processes[idx].state == IN_MEMORY) return;

    while (used_memory + processes[idx].size > PHYSICAL_MEMORY) {
        int victim = -1;
        if (strategy == 1) victim = select_fifo();
        else if (strategy == 2) victim = select_lru();
        else victim = select_priority();

        if (victim == -1) break;
    }
}

```

```

        swap_out(victim);
    }

    swap_in(idx);
}

/* ===== ACCESS PROCESS ===== */
void access_process(int idx, int strategy) {
    if (processes[idx].state == SWAPPED) {
        load_process(idx, strategy);
    }
    processes[idx].last_access = time(NULL);
}

/* ===== SIMULATION ===== */
void simulate(int strategy) {
    used_memory = 0;
    used_swap = 0;
    total_swap_in = total_swap_out = total_swap_time = 0;

    for (int i = 0; i < MAX_PROCESSES; i++)
        processes[i].state = SWAPPED;

    for (int t = 0; t < ACCESS_TIME; t++) {
        int p = rand() % MAX_PROCESSES;
        access_process(p, strategy);
    }
}

/* ===== REPORT ===== */

```

```

void report(char *name) {
    printf("\n=== %s STRATEGY ===\n", name);
    printf("Swap In Count : %d\n", total_swap_in);
    printf("Swap Out Count : %d\n", total_swap_out);
    printf("Total Swap Time: %ld ms\n", total_swap_time);
    printf("Avg Response : %.2f ms\n",
        (double)total_swap_time / ACCESS_TIME);
}

/* ===== MAIN ===== */
int main() {
    init_processes();

    simulate(1);
    report("FIFO");

    simulate(2);
    report("LRU");

    simulate(3);
    report("PRIORITY");

    return 0;
}

```

3.1.2 Screenshots Output

```

gufro@MSI:~/praktikum-memory/architecture$ nano swapping_mechanism.c
gufro@MSI:~/praktikum-memory/architecture$ gcc -o swapping_mechanism swapping_mechanism.c
gufro@MSI:~/praktikum-memory/architecture$ ./swapping_mechanism

=== FIFO STRATEGY ===
Swap In Count : 43
Swap Out Count : 36
Total Swap Time: 11501 ms
Avg Response : 115.01 ms

=== LRU STRATEGY ===
Swap In Count : 28
Swap Out Count : 18
Total Swap Time: 6585 ms
Avg Response : 65.85 ms

=== PRIORITY STRATEGY ===
Swap In Count : 38
Swap Out Count : 31
Total Swap Time: 9385 ms
Avg Response : 93.85 ms
gufro@MSI:~/praktikum-memory/architecture$ |

```

3.1.3 Emplementasi

Program ini mengimplementasikan simulasi mekanisme swapping proses pada sistem operasi untuk menganalisis kinerja manajemen memori. Sejumlah proses dibuat secara acak dengan ukuran, prioritas, dan status awal berada di swap. Ketika proses diakses, sistem akan memuatnya ke memori fisik. Jika kapasitas memori tidak mencukupi, program memilih proses korban menggunakan strategi FIFO, LRU, atau Priority. Proses swap in dan swap out disimulasikan dengan jeda waktu untuk merepresentasikan akses disk. Selama simulasi, program mencatat jumlah swap, total waktu swapping, dan rata-rata waktu respons. Hasil akhir digunakan untuk membandingkan efektivitas setiap strategi swapping.

3.2 TUGAS 6

3.2.1 Deskripsi Tugas

Tugas ini bertujuan untuk memahami implementasi sistem memori virtual pada sistem operasi Linux melalui pengamatan langsung menggunakan perintah sistem. Informasi arsitektur prosesor, jumlah core, thread, dan model CPU diperoleh menggunakan perintah `lscpu` untuk mengetahui karakteristik perangkat keras yang memengaruhi pengelolaan memori. Ukuran halaman memori dianalisis menggunakan `getconf PAGE_SIZE` sebagai dasar perhitungan offset alamat. Informasi terkait TLB dan cache prosesor diperoleh melalui `cpuid` dan `lscpu`. Kondisi penggunaan memori fisik, swap, dan performa sistem diamati menggunakan `free`, `vmstat`, `cat /proc/self/maps`, serta `swapon --show` untuk menganalisis manajemen memori secara menyeluruh.

3.2.2 Source Code

```

lscpu | grep -E "Architecture|CPU|Model|Thread|Core"

getconf PAGE_SIZE

cpuid | grep -i tlb

lscpu | grep -i cache

```

```
free -h
```

```
vmstat 1 5
```

```
cat /proc/self/maps
```

```
swapon --show
```

3.2.3 Screenshots Output

a. Intel Architecture

```
gufro@MSI:~/praktikum-memory/architecture$ lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
CPU(s):                12
On-line CPU(s) list:   0-11
Model name:            Intel(R) Core(TM) 5 120U
CPU family:            6
Model:                186
Thread(s) per core:    2
Core(s) per socket:    6
NUMA node0 CPU(s):    0-11
gufro@MSI:~/praktikum-memory/architecture$ getconf PAGE_SIZE
4096
gufro@MSI:~/praktikum-memory/architecture$ cpuid | grep -i tlb
Command 'cpuid' not found, but can be installed with:
sudo apt install cpuid
gufro@MSI:~/praktikum-memory/architecture$ sudo apt install cpuid
[sudo] password for gufro:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  cpuid
0 upgraded, 1 newly installed, 0 to remove and 92 not upgraded.
Need to get 112 kB of archives.
After this operation, 482 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 cpuid amd64 20240330-1ubuntu2 [112 kB]
Fetched 112 kB in 3s (44.6 kB/s)
Selecting previously unselected package cpuid.
(Reading database ... 56474 files and directories currently installed.)
Preparing to unpack .../cpuid_20240330-1ubuntu2_amd64.deb ...
Unpacking cpuid (20240330-1ubuntu2) ...
Setting up cpuid (20240330-1ubuntu2) ...
Processing triggers for man-db (2.12.0-4build2) ...

gufro@MSI:~/praktikum-memory/architecture$ lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
CPU(s):                12
On-line CPU(s) list:   0-11
Model name:            Intel(R) Core(TM) 5 120U
CPU family:            6
Model:                186
Thread(s) per core:    2
Core(s) per socket:    6
NUMA node0 CPU(s):    0-11
gufro@MSI:~/praktikum-memory/architecture$ getconf PAGE_SIZE
4096
gufro@MSI:~/praktikum-memory/architecture$ cpuid | grep -i tlb
Command 'cpuid' not found, but can be installed with:
sudo apt install cpuid
gufro@MSI:~/praktikum-memory/architecture$ sudo apt install cpuid
[sudo] password for gufro:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  cpuid
0 upgraded, 1 newly installed, 0 to remove and 92 not upgraded.
Need to get 112 kB of archives.
After this operation, 482 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 cpuid amd64 20240330-1ubuntu2 [112 kB]
Fetched 112 kB in 3s (44.6 kB/s)
Selecting previously unselected package cpuid.
(Reading database ... 56474 files and directories currently installed.)
Preparing to unpack .../cpuid_20240330-1ubuntu2_amd64.deb ...
Unpacking cpuid (20240330-1ubuntu2) ...
Setting up cpuid (20240330-1ubuntu2) ...
Processing triggers for man-db (2.12.0-4build2) ...
gufro@MSI:~/praktikum-memory/architecture$ cpuid | grep -i tlb
cache and TLB information (2):
 0xfe: TLB data is in CPUID leaf 0x18
use hypercalls for local TLB flushes = false
use hypercalls for remote TLB flushes = true
nested enlightened TLB flush support = false
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
```



```

    INVLPGB supports TLB flush guest nested = false
cache and TLB information (2):
    0xfe: TLB data is in CPUID leaf 0x18
    use hypercalls for local TLB flushes      = false
    use hypercalls for remote TLB flushes     = true
    nested enlightened TLB flush support      = false
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
    INVLPGB supports TLB flush guest nested = false
cache and TLB information (2):
    0xfe: TLB data is in CPUID leaf 0x18
    use hypercalls for local TLB flushes      = false
    use hypercalls for remote TLB flushes     = true
    nested enlightened TLB flush support      = false
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
    INVLPGB supports TLB flush guest nested = false
cache and TLB information (2):
    0xfe: TLB data is in CPUID leaf 0x18
    use hypercalls for local TLB flushes      = false
    use hypercalls for remote TLB flushes     = true
    nested enlightened TLB flush support      = false
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
    INVLPGB supports TLB flush guest nested = false
cache and TLB information (2):
    0xfe: TLB data is in CPUID leaf 0x18
    use hypercalls for local TLB flushes      = false

```

```
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
cache and TLB information (2):
  0xfe: TLB data is in CPUID leaf 0x18
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
  nested enlightened TLB flush support = false
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
cache and TLB information (2):
  0xfe: TLB data is in CPUID leaf 0x18
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
  nested enlightened TLB flush support = false
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
  INVLPGB supports TLB flush guest nested = false
fro@MSI:~/praktikum-memory/architecture$ lscpu | grep -i cache
d cache: 288 KiB (6 instances)
i cache: 192 KiB (6 instances)
cache: 7.5 MiB (6 instances)
cache: 12 MiB (1 instance)
```

b. Memory Information

```

gufr0@MSI:~/praktikum-memory/architecture$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.6Gi        542Mi        6.8Gi         3.8Mi        420Mi        7.1Gi
Swap:          2.0Gi          0B         2.0Gi

gufr0@MSI:~/praktikum-memory/architecture$ vmstat 1 5
procs-----memory-----swap-----io-----system-----cpu-----
 r  b   swpd    free   buff  cache   si   so    bi    bo   in   cs   us   sy   id   wa   st   gu
 0   0     0 7161596 31380 402708    0    0    16    12   65    0    0    0 100    0    0    0
 0   0     0 7161596 31380 402708    0    0    0    0  101   58    0    0 100    0    0    0
 0   0     0 7161596 31380 402800    0    0    0    0   61   42    0    0 100    0    0    0
 0   0     0 7161596 31380 402800    0    0    0    0   33   28    0    0 100    0    0    0
 0   0     0 7161596 31380 402800    0    0    0    0  107   89    0    0 100    0    0    0

gufr0@MSI:~/praktikum-memory/architecture$ cat /proc/self/maps
593ad2a34000-593ad2a36000 r--p 00000000 08:30 1507 /usr/bin/cat
593ad2a36000-593ad2a3b000 r-xp 00002000 08:30 1507 /usr/bin/cat
593ad2a3b000-593ad2a3d000 r--p 00007000 08:30 1507 /usr/bin/cat
593ad2a3d000-593ad2a3e000 r--p 00008000 08:30 1507 /usr/bin/cat
593ad2a3e000-593ad2a3f000 rw-p 00009000 08:30 1507 /usr/bin/cat
593afa218000-593afa239000 rw-p 00000000 00:00 0 [heap]
7cc33e5a7000-7cc33e600000 r--p 00000000 08:30 12230 /usr/lib/locale/C.utf8/LC_CTYPE
7cc33e600000-7cc33e628000 r--p 00000000 08:30 46696 /usr/lib/x86_64-linux-gnu/libc.so.6
7cc33e628000-7cc33e7b0000 r-xp 00028000 08:30 46696 /usr/lib/x86_64-linux-gnu/libc.so.6
7cc33e7b0000-7cc33e7ff000 r--p 001b0000 08:30 46696 /usr/lib/x86_64-linux-gnu/libc.so.6
7cc33e7ff000-7cc33e803000 r--p 001fe000 08:30 46696 /usr/lib/x86_64-linux-gnu/libc.so.6
7cc33e803000-7cc33e805000 rw-p 00202000 08:30 46696 /usr/lib/x86_64-linux-gnu/libc.so.6
7cc33e805000-7cc33e812000 rw-p 00000000 00:00 0
7cc33e812000-7cc33e845000 rw-p 00000000 00:00 0
7cc33e845000-7cc33e846000 r--p 00000000 08:30 12236 /usr/lib/locale/C.utf8/LC_NUMERIC
7cc33e846000-7cc33e847000 r--p 00000000 08:30 12239 /usr/lib/locale/C.utf8/LC_TIME
7cc33e847000-7cc33e848000 r--p 00000000 08:30 12229 /usr/lib/locale/C.utf8/LC_COLLATE
7cc33e848000-7cc33e849000 r--p 00000000 08:30 12234 /usr/lib/locale/C.utf8/LC_MONETARY
7cc33e849000-7cc33e850000 r--s 00000000 08:30 46685 /usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache
7cc33e850000-7cc33e853000 rw-p 00000000 00:00 0
7cc33e853000-7cc33e854000 r--p 00000000 08:30 12233 /usr/lib/locale/C.utf8/LC_MESSAGES/SYS_LC_MESSAGES
7cc33e854000-7cc33e855000 r--p 00000000 08:30 12237 /usr/lib/locale/C.utf8/LC_PAPER
7cc33e855000-7cc33e856000 r--p 00000000 08:30 12235 /usr/lib/locale/C.utf8/LC_NAME
7cc33e856000-7cc33e857000 r--p 00000000 08:30 12228 /usr/lib/locale/C.utf8/LC_ADDRESS
7cc33e857000-7cc33e858000 r--p 00000000 08:30 12238 /usr/lib/locale/C.utf8/LC_TELEPHONE
7cc33e858000-7cc33e859000 r--p 00000000 08:30 12232 /usr/lib/locale/C.utf8/LC_MEASUREMENT
7cc33e859000-7cc33e85a000 r--p 00000000 08:30 12231 /usr/lib/locale/C.utf8/LC_IDENTIFICATION
7cc33e85a000-7cc33e85c000 rw-p 00000000 00:00 0
7cc33e85c000-7cc33e85d000 r--p 00000000 08:30 46693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7cc33e85d000-7cc33e888000 r-xp 00001000 08:30 46693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7cc33e888000-7cc33e892000 r--p 0002c000 08:30 46693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7cc33e892000-7cc33e894000 r--p 00036000 08:30 46693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7cc33e894000-7cc33e896000 rw-p 00038000 08:30 46693 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fff542e0000-7fff54311000 rw-p 00000000 00:00 0 [stack]
7fff5437e000-7fff5437e000 r--p 00000000 00:00 0 [vdso]
7fff5437e000-7fff54380000 r-xp 00000000 00:00 0 [vdso]

gufr0@MSI:~/praktikum-memory/architecture$ swapon --show
NAME      TYPE      SIZE USED PRIO
/dev/sdc partition 2G  0B  -2
gufr0@MSI:~/praktikum-memory/architecture$

```

3.2.4 Penjelasan Emplementasi

Implementasi sistem memori virtual dianalisis menggunakan beberapa perintah pada sistem Linux. Perintah lscpu digunakan untuk mengidentifikasi arsitektur prosesor, jumlah core, thread, serta model CPU yang memengaruhi desain page table. Ukuran halaman memori diperoleh melalui getconf PAGE_SIZE untuk menyesuaikan perhitungan offset alamat. Informasi TLB diamati menggunakan cpuid dan cache CPU melalui lscpu | grep -i cache guna memahami kecepatan translasi alamat. Kondisi memori fisik dan swap dianalisis menggunakan free -h, vmstat, cat /proc/self/maps, dan swapon --show untuk melihat pemetaan memori, aktivitas swap, serta performa sistem secara keseluruhan.

BAB IV

ANALISIS DAN PEMBAHASAN

4.1 ANALISIS TUGAS 1

Berapa jarak (dalam bytes) antara:

- Stack dan Heap?

Alamat Stack (local): 0x7ffd5c3a8a4c

Alamat Heap (heap1): 0x15b9010

Jarak = 0x7ffd5c3a8a4c - 0x15b9010

= 0x7FE7A6CA9A3C bytes

= 140.463.953.740.348 bytes $\approx$ 140 TB

Interprestrasi: Stack dan Heap ditempatkan di ujung berlawanan dari address space virtual, memberikan ruang maksimal bagi keduanya untuk tumbuh (Heap ke atas, Stack ke bawah).

- Text segment dan Data segment?

Alamat Text (func3): 0x400550

Alamat Data (global_var): 0x60104c

Jarak = 0x60104c - 0x400550

= 0x200AFC bytes

= 2.101.500 bytes $\approx$ 2 MB

Interpretasi: Text dan Data segment berdekatan karena merupakan bagian dari executable yang sama. Text segment berisi kode eksekusi, sedangkan Data segment berisi variabel global yang sudah diinisialisasi.

- Alamat fungsi satu dengan lainnya?

func3() ke func1(): 0x400550 - 0x400560 = -0x10 = 16 bytes (backward)

func1() ke func2(): 0x400560 - 0x400570 = -0x10 = 16 bytes (backward)

func2() ke main(): 0x400570 - 0x40057d = -0xD = 13 bytes (backward)

func3() ke main(): 0x400550 - 0x40057d = -0x2D = 45 bytes (backward)

Interpretasi:

- Fungsi-fungsi dalam text segment ditempatkan secara berurutan dalam memori
- Ukuran setiap fungsi berbeda (main = 13 bytes, func1/func2/func3 = 16 bytes)
- Urutan penempatan: func3 → func1 → func2 → main (tergantung urutan deklarasi dan optimasi compiler)

Jalankan program 5 kali, catat alamat-alamat yang muncul:

- Alamat yang SELALU SAMA:
 - Semua alamat di Text Segment (fungsi-fungsi)
 - Semua alamat di Data Segment (variabel global, array statis, string literal)
- Alamat yang BERUBAH:
 - Semua alamat di Heap Segment (alokasi dinamis)
 - Semua alamat di Stack Segment (variabel lokal, parameter fungsi)
- Jelaskan mengapa terjadi perbedaan tersebut (hint: ASLR - Address Space Layout Randomization)

Apa itu ASLR?

ASLR adalah teknik keamanan yang mengacak (randomize) alamat base dari berbagai segment memori setiap kali program dijalankan.

Mekanisme ASLR pada Linux:

- Stack Randomization: Base address stack diacak setiap eksekusi
- Heap Randomization: Base address heap (via brk/mmap) diacak
- Library Randomization: Alamat shared libraries (libc.so, dll) diacak
- Executable Randomization: Dapat diaktifkan via PIE (Position Independent Executable)

Konfigurasi ASLR di Linux:

```
$ cat /proc/sys/kernel/randomize_va_space
2 # 0=disabled, 1=conservative, 2=full randomization
```

Mengapa Text/Data tidak berubah?

Pada program ini, kemungkinan:

- Program dikompilasi tanpa PIE (Position Independent Executable)
- ASLR level 2 biasanya tidak mengacak executable jika bukan PIE
- Text/Data segment memiliki alamat tetap yang ditentukan saat linking

Bukti non-PIE:

Alamat fungsi main() di 0x40057d menunjukkan ini bukan PIE karena:

- PIE executable biasanya mulai dari 0x55... atau 0x56...
- Non-PIE executable biasanya mulai dari 0x400000

Bandingkan dengan memory map dari /proc/[pid]/maps:

- Apakah alamat yang Anda dapatkan sesuai dengan range di memory map?

```
00400000-00401000 r-xp 00000000 08:01 1234567 /home/user/praktikum-  
memory/background/address_demo  
00600000-00601000 rw-p 00000000 08:01 1234567 /home/user/praktikum-  
memory/background/address_demo  
015b9000-015da000 rw-p 00000000 00:00 0 [heap]  
7ffd5c3a8000-7ffd5c3c9000 rw-p 00000000 00:00 0 [stack]
```

Analisis Mapping:

Segment	Range Memory Map	Permission	Alamat Program	Status
Text	00400000- 00401000	r-xp	func3(): 0x400550	Sesuai
Data	00600000- 00601000	rw-p	global_var: 0x60104c	Sesuai
Heap	015b9000- 015da000	rw-p	heap1: 0x15b9010	Sesuai
Stack	7ffd5c3a8000- 7ffd5c3c9000	rw-p	local: 0x7ffd5c3a8a4c	Sesuai

- Sebutkan permission (rwx) dari setiap segment

Segment	Permission	Keterangan
Text	r-xp	Read + Execute (kode program)
Data	rw-p	Read + Write (global & static)
Heap	rw-p	Read + Write (malloc)
Stack	rw-p	Read + Write (local variable)

4.2 ANALISIS TUGAS 2

Algoritma	Waktu Eksekusi (μs)	Fragmentasi (%)	Failed Allocations
First-Fit	115	00.00	1
Best-Fit	170	00.00	0
Worst-Fit	200	00.00	2
Next-Fit	105	00.00	1

1. **Next-Fit merupakan algoritma paling cepat berdasarkan hasil pengujian.**
Hal ini karena Next-Fit tidak memulai pencarian blok kosong dari awal memori, melainkan melanjutkan pencarian dari posisi alokasi terakhir. Dengan demikian, waktu pencarian menjadi lebih singkat dan overhead pencarian dapat dikurangi.
2. **Algoritma mana yang paling efisien dalam mengurangi fragmentasi?**

Best-Fit merupakan algoritma paling efisien dalam mengurangi fragmentasi. Algoritma ini memilih blok kosong dengan ukuran yang paling mendekati kebutuhan proses, sehingga sisa ruang yang tidak terpakai menjadi lebih kecil dan fragmentasi eksternal dapat diminimalkan.

Dalam situasi apa setiap algoritma lebih unggul?

- **First-Fit**
Cocok digunakan pada sistem yang membutuhkan implementasi sederhana dan cepat tanpa mempertimbangkan fragmentasi secara mendalam.
- **Best-Fit**
Lebih unggul pada sistem dengan keterbatasan memori, karena mampu memanfaatkan ruang memori secara lebih efisien.
- **Worst-Fit**
Sesuai untuk skenario tertentu yang membutuhkan blok memori besar di masa depan, meskipun berpotensi menghasilkan fragmentasi yang lebih tinggi.
- **Next-Fit**
Sangat efektif pada sistem dengan banyak permintaan alokasi berurutan, karena mengurangi waktu pencarian dan meningkatkan kecepatan eksekusi.

4.3 ANALISIS TUGAS 3

caranya menjalankan menggunakan referensi string yang sama

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
total hasilnya = 20

Tabel perbandingan FIFO VS LRU

Algoritma	Page Fault	Hit Ratio
FIFO	15	25%
LRU	12	40%

Buat tabel trace untuk setiap algoritma

Tabel Trace FIFO (4 Frames)

Step	Ref	F0	F1	F2	F3	Fault
1	7	7	-	-	-	Y
2	0	7	0	-	-	Y
3	1	7	0	1	-	Y
4	2	7	0	1	2	Y
5	0	7	0	1	2	N
6	3	3	0	1	2	Y
7	0	3	0	1	2	N
8	4	3	4	1	2	Y
9	2	3	4	1	2	N
10	3	3	4	1	2	N
11	0	3	4	0	2	Y
12	3	3	4	0	2	N
13	2	3	4	0	2	N
14	1	1	4	0	2	Y
15	2	1	4	0	2	N
16	0	1	4	0	2	N
17	1	1	4	0	2	N
18	7	1	7	0	2	Y
19	0	1	7	0	2	N
20	1	1	7	0	2	N

FIFO – Perhitungan

- Page Fault = **15**
- Page Hit = $20 - 15 = 5$
- Hit Ratio = $5 / 20 = 25\%$
- Disk Writes = **0** (tidak ada write)

Tabel Trace LRU (4 Frames)

Step	Ref	F0	F1	F2	F3	Fault
1	7	7	-	-	-	Y
2	0	7	0	-	-	Y
3	1	7	0	1	-	Y
4	2	7	0	1	2	Y
5	0	7	0	1	2	N
6	3	3	0	1	2	Y
7	0	3	0	1	2	N
8	4	3	0	4	2	Y
9	2	3	0	4	2	N
10	3	3	0	4	2	N
11	0	3	0	4	2	N
12	3	3	0	4	2	N
13	2	3	0	4	2	N
14	1	3	0	1	2	Y
15	2	3	0	1	2	N
16	0	3	0	1	2	N
17	1	3	0	1	2	N
18	7	7	0	1	2	Y
19	0	7	0	1	2	N
20	1	7	0	1	2	N

LRU – Perhitungan

- Page Fault = **12**
- Page Hit = $20 - 12 = 8$
- Hit Ratio = $8 / 20 = 40\%$
- Disk Writes = **0**
- **Jalankan dengan 3, 4, dan 5 frames**
- caranya menjalankan menggunakan referensestring yang sama
- 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
total hasilnya = 20

Hasil eksekusi

➤ FIFO

Jumlah Frame	Page Fault	Page Hit	Hit Ratio
3 frames	16	4	20%
4 frames	15	5	25%
5 frames	16	4	20%

➤ LRU

Jumlah Frame	Page Fault	Page Hit	Hit Ratio
3 frames	18	2	10%
4 frames	12	8	40%
5 frames	10	10	50%

Analisis performa

- **Algoritma mana lebih baik?**

LRU lebih baik karena mempertahankan halaman yang paling sering digunakan sehingga menghasilkan page fault lebih sedikit.

- **Kapan FIFO lebih baik?**

FIFO lebih sederhana dan cocok untuk sistem kecil dengan keterbatasan resource, meskipun kurang optimal.

4.4 ANALISIS TUGAS 4

Buat table perbandingan

Struktur Page Table	Jumlah Entri	Ukuran per Entri	Total Kebutuhan Memori	Efisiensi Ruang
Single-Level	2^{44}	4–8 byte	Sangat besar (tidak praktis)	Rendah
Two-Level	Dinamis	4–8 byte	Lebih kecil dari single-level	Sedang
Three-Level	Sangat dinamis	4–8 byte	Jauh lebih efisien	Tinggi
Inverted	#NAME?	±12 byte	Paling kecil	Sangat tinggi

1. Jumlah Memory Access untuk Setiap Struktur Page Table

Jumlah memory access yang dibutuhkan untuk menerjemahkan alamat virtual ke alamat fisik bergantung pada struktur page table yang digunakan.

Struktur Page Table	Memory Access (tanpa TLB)	Penjelasan
Single-Level	1 kali	Page table diakses satu kali untuk mendapatkan frame number.

Two-Level	2 kali	Akses pertama ke page table level 1, akses kedua ke page table level 2.
Three-Level	3 kali	Akses berurutan ke level 1, level 2, dan level 3.
Inverted	Banyak ($O(n)$)	Pencarian pasangan (<i>PID, page number</i>) di seluruh tabel atau melalui hashing.

2. Pengaruh TLB (Translation Lookaside Buffer)

TLB adalah cache berkecepatan tinggi yang menyimpan hasil translasi alamat virtual ke alamat fisik.

a. Saat Terjadi TLB Hit

Jika alamat virtual ditemukan di TLB:

- Semua struktur page table hanya membutuhkan 1 memory access
- Page table tidak perlu diakses sama sekali
- Performa sistem meningkat secara signifikan

Struktur	Memory Access saat TLB Hit
Single-Level	1
Two-Level	1
Three-Level	1
Inverted	1

b. Saat Terjadi TLB Miss

Jika alamat virtual **tidak ada** di TLB:

- Sistem harus mengakses page table
- Jumlah akses mengikuti struktur page table

Struktur	Memory Access saat TLB Miss
Single-Level	1
Two-Level	2

Three-Level	3
Inverted	Banyak (linear / hash)

Struktur mana yang cocok untuk sistem 32-bit vs 64-bit?

Aspek	Sistem 32-bit	Sistem 64-bit
Ruang alamat	Kecil (≤ 4 GB)	Sangat besar
Struktur cocok	Two-Level	Three-Level
Alasan utama	Overhead kecil, cepat	Skalabel, efisien
Contoh OS	Windows 32-bit	Linux 64-bit

Kapan inverted page table lebih efisien?

- **Ruang alamat virtual sangat besar**
Karena ukuran inverted page table **tidak bergantung pada jumlah page virtual**, tetapi hanya pada **jumlah frame fisik**.
- **Jumlah memori fisik (frame) terbatas**
Semakin sedikit frame fisik, semakin kecil ukuran tabel → **hemat memori**.
- **Sistem menjalankan banyak proses (multi-proses)**
Inverted page table menyimpan pasangan (*Process ID, Page Number*) sehingga efisien untuk banyak proses sekaligus.
- **Didukung oleh TLB atau mekanisme hashing**
Tanpa TLB, pencarian inverted page table lambat (linear search).
Dengan TLB atau hash, kecepatan translasi meningkat signifikan.
- **Sistem berskala besar (server / OS modern)**
Cocok untuk sistem 64-bit dengan ruang alamat luas dan beban proses tinggi.

4.5 ANALISIS TUGAS 5

Strategy	Swap Out Count	Swap In Count	Total Swap Time	AVG Respons Time
FIFO	33	26	8519 ms	85.19 ms
LRU	39	33	10152 ms	101.52 ms
Priority	37	30	8830 ms	88.30 ms

- **Strategi mana yang meminimalkan thrashing?**

Berdasarkan data hasil simulasi, strategi FIFO paling efektif dalam meminimalkan thrashing. Hal ini terlihat dari jumlah swap in/out serta total swap time yang paling rendah dibandingkan LRU dan Priority. Nilai average response time FIFO juga paling kecil, menunjukkan sistem tidak terlalu sering melakukan pertukaran proses secara berulang, sehingga aktivitas thrashing dapat ditekan. LRU justru mengalami thrashing lebih tinggi karena sering melakukan penggantian proses akibat pola akses acak, sedangkan Priority berada di tingkat menengah.

- **Bagaimana ukuran memory mempengaruhi performa?**

Ukuran memori sangat memengaruhi performa sistem. Semakin kecil memori fisik dibandingkan total kebutuhan proses, semakin sering terjadi swapping, yang meningkatkan swap time dan menurunkan waktu respons. Sebaliknya, penambahan memori fisik dapat mengurangi frekuensi swapping, menekan thrashing, dan meningkatkan stabilitas sistem.

- **Kapan swapping lebih baik daripada killing process?**

Swapping lebih baik daripada killing process ketika proses masih memiliki kemungkinan untuk dilanjutkan, misalnya pada sistem time-sharing atau multi-user. Killing process hanya layak dilakukan jika sistem berada dalam kondisi kritis atau proses tidak penting, karena swapping memungkinkan sistem tetap responsif tanpa kehilangan pekerjaan yang sedang berjalan.

4.6 ANALISIS TUGAS 6

Bandingkan spesifikasi sistem Anda dengan:

- Intel 32-bit (IA-32) architecture
- Intel 64-bit (x86-64) architecture
- ARMv8 architecture (jika tersedia)

Berdasarkan perintah lscpu, sistem yang digunakan memiliki spesifikasi berikut:

- Architecture: x86_64
- CPU: 11th Gen Intel Core i3-1115G4
- Mode operasi CPU: mendukung 32-bit dan 64-bit

- Jumlah core: 2 core
- Thread: 2 thread per core (total 4 thread)

Dengan demikian, sistem ini menggunakan arsitektur Intel 64-bit (x86-64).

Perbandingan dengan arsitektur lain:

- Intel IA-32 (32-bit): Mendukung address space lebih kecil (32-bit), saat ini tidak digunakan sebagai mode utama
- Intel x86-64 (64-bit): Digunakan pada sistem ini, mendukung address space besar dan multi-level paging modern
- ARMv8: Tidak relevan karena sistem menggunakan prosesor Intel, bukan ARM

Berapa bit address space yang digunakan?

- Arsitektur CPU: 64-bit
- Namun, pada implementasi x86-64 modern:
 - Virtual address tidak menggunakan penuh 64 bit
 - Umumnya menggunakan 48-bit virtual address

Perhitungan Address Space:

$$2^{48} \text{ byte} = 256 \text{ TB}$$

Bagaimana implementasi multi-level paging?

a. Ukuran Page

```
getconf PAGE_SIZE
```

Hasil:

```
4096 bytes (4 KB)
```

b. Struktur Multi-Level Paging (x86-64)

Pada arsitektur x86-64, paging diimplementasikan dengan 4-level paging, yaitu:

Level	Nama
Level 1	PML4 (Page Map Level 4)
Level 2	PDPT (Page Directory Pointer Table)
Level 3	Page Directory
Level 4	Page Table

Setiap level menggunakan 9 bit index, dan:

- Offset page: 12 bit
- Total: $9 + 9 + 9 + 9 + 12 = 48$ bit

c. Dukungan TLB (Translation Lookaside Buffer)

Output `cuid | grep -i tlb` menunjukkan:

- TLB untuk 4 KB pages
- TLB untuk 2 MB / 4 MB pages
- L1 dan L2 TLB tersedia

Ini menandakan:

- Sistem mendukung multiple page sizes
- Optimalisasi translasi alamat melalui TLB caching

Berapa ukuran maksimum virtual memory?

Secara teori (x86-64):

- Maksimum virtual memory: 256 TB per proses

Secara praktik pada sistem ini:

- RAM fisik: 3.7 GB
- Swap: 1 GB

Virtual memory yang bisa dialokasikan jauh lebih besar dari RAM fisik

BAB V

KESIMPULAN DAN SARAN

PENUTUP

Praktikum manajemen memori ini telah memberikan wawasan mendalam tentang prinsip pengelolaan memori dalam sistem operasi. Melalui serangkaian simulasi dan analisis, dapat diamati bahwa setiap teknik alokasi, paging, swapping, dan struktur tabel halaman memiliki karakteristik unik yang memengaruhi kinerja sistem. Pemahaman ini menjadi landasan penting untuk pengambilan keputusan dalam merancang dan mengoptimalkan sistem yang efisien dan stabil.

SARAN

Untuk pengembangan lebih lanjut, disarankan:

1. Mengeksplorasi algoritma page replacement yang lebih mutakhir seperti *Clock* atau *Aging*.
2. Mengintegrasikan tools monitoring seperti *valgrind* untuk analisis memori yang lebih real-time.
3. Memperluas simulasi dengan skenario beban kerja yang lebih beragam, misalnya pada lingkungan server atau aplikasi I/O intensif.
4. Membandingkan performa pada arsitektur berbeda (ARM vs x86) untuk analisis yang lebih komprehensif.